

SAPIENCE · SECURITY REVIEW

Sapience Review.

A security review of Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Sapience	Review	Guardian	July 16 – August 19, 2025	August 26, 2025

§ 00

Contents.

130 pages — overview, confidence ranking, scope, 87 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 74 findings	P. 13
	Remediation Review · 13 findings	P. 111
§ 05	Disclaimer	P. 129
§ 06	About Guardian	P. 130

§ 01

Overview.

Guardian reviewed Review across 2 review rounds. The final report includes 8 High, 18 Medium, 30 Low, 31 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	sapiencexyz/sapience
CLIENT	METHODOLOGY
Sapience	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Review	dc67864b4e70
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	a1d13fda0368
AUDIT WINDOW	
July 16 – August 19, 2025	
FINAL REPORT	
August 26, 2025	

1.1 Confidence Ranking

2 / 5

Low Confidence

Guardian assigns a Confidence Ranking of 2 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	8	0	3	0	5
Medium	18	0	12	0	6
Low	30	0	23	0	7
Informational	31	0	11	1	19
All severities	87	0	49	1	37

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence ASSIGNED	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Review in `sapiencexyz/sapience`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Persistent Withdrawal Intent Blocks Subsequent Bond Withdrawals	Logical Error	HIGH	RESOLVED	p. 13
H-02	Manual Settlement Yields Fractional/Innacurate Payouts	Logical Error	HIGH	RESOLVED	p. 15
H-03	Zero-Price Settlement Can Create Immediate Bad Debt in Binary Markets	Logical Error	HIGH	RESOLVED	p. 17
H-04	depositBond Function Can Be Abused To Drain Bridge ETH	Validation	HIGH	RESOLVED	p. 19
H-05	Total Loss For LPs In Binary Markets	Logical Error	HIGH	ACKNOWLEDGED	p. 21
H-06	Insufficient Slippage When Closing LPs	MEV	HIGH	RESOLVED	p. 22
M-01	Protocol Is Vulnerable To Just In Time Liquidity	Warning	MEDIUM	ACKNOWLEDGED	p. 24
M-02	market.sqrtPriceMaxX96 Is Increased By Tick Spacing	Warning	MEDIUM	ACKNOWLEDGED	p. 26
M-03	Unreduced Debt Following Partial Liquidity Withdrawals Amplifies Impermanent Loss Exposure	Warning	MEDIUM	ACKNOWLEDGED	p. 28
M-04	Dust-1-wei Debt Can Make closeLiquidityPosition Revert	Rounding	MEDIUM	RESOLVED	p. 30
M-05	Race Condition in Bridged Bond Withdrawal	Race Condition	MEDIUM	RESOLVED	p. 33

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-06	Risk Of assertionId Collision In UMAOracle	Logical Error	MEDIUM	ACKNOWLEDGED	p. 35
M-07	Arbitrary Asserter Allows Bond Theft	Logical Error	MEDIUM	ACKNOWLEDGED	p. 37
M-08	Incorrect Market Params Used	Logical Error	MEDIUM	RESOLVED	p. 39
M-09	Slippage Should Be Applied For Trading Positions	MEV	MEDIUM	ACKNOWLEDGED	p. 41
M-10	Repeated Liquidity Decrease Reverts Due Rounding	Logical Error	MEDIUM	RESOLVED	p. 43
M-11	Market Settlement May Be Blocked	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 45
M-12	Manual Settlement May Be Manipulated	MEV	MEDIUM	RESOLVED	p. 46
M-13	Race Condition May Block The Settlement	Race Condition	MEDIUM	ACKNOWLEDGED	p. 47
M-14	FeeCollectorNFT Enables Under-collateralized LP Actions	Warning	MEDIUM	RESOLVED	p. 48
M-15	Market Creations DoS'ed By Existing Pool	DoS	MEDIUM	ACKNOWLEDGED	p. 49
L-01	Incompatibility With Non Standard ERC20 Tokens	Best Practices	LOW	RESOLVED	p. 50
L-02	Missing Price Checks For Binary Markets Settlement	Validation	LOW	ACKNOWLEDGED	p. 51
L-03	Missing Multiple UMA Basic Checks Upon Settlement	Validation	LOW	ACKNOWLEDGED	p. 52
L-04	Opaque Reverts in Swap Functions During Insufficient Liquidity Scenarios	Validation	LOW	RESOLVED	p. 54
L-05	Excessive Gas Allocation for LayerZero Balance Updates	Configuration	LOW	ACKNOWLEDGED	p. 55
L-06	UMA Bridge Owner Can Influence Settlements	Trust Assumptions	LOW	ACKNOWLEDGED	p. 56

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-07	Dispute Is Blocked When Contract Is Underfunded	Logical Error	LOW	ACKNOWLEDGED	p. 57
L-08	Binary Markets Cannot Resolve Early	Warning	LOW	ACKNOWLEDGED	p. 58
L-09	Ambiguous Token Data	Configuration	LOW	ACKNOWLEDGED	p. 59
L-10	Market Group Initialized With Arbitrary Nonce	Events	LOW	ACKNOWLEDGED	p. 60
L-11	Potential UMA Price Manipulation By Whales	Validation	LOW	ACKNOWLEDGED	p. 61
L-12	Gas Thresholds Are Checked After Sending The LZ Message	Events	LOW	RESOLVED	p. 62
L-13	UMA Callbacks Fail After Manual Settlement	Unexpected Behavior	LOW	RESOLVED	p. 63
L-14	Duplicate Event Emission During Balance Update	Events	LOW	RESOLVED	p. 64
L-15	Timestamp is not enforced in UMA claim	Best Practices	LOW	ACKNOWLEDGED	p. 65
L-16	FeeCollectorNFT Is Transferrable And Can Not Be Burned	Trust Assumptions	LOW	RESOLVED	p. 66
L-17	Changing The NonFungiblePositionManager Address Breaks The Protocol	Configuration	LOW	ACKNOWLEDGED	p. 67
L-18	Trading Is Allowed Before Market Start Time	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 68
L-19	Insufficient Minimum Collateral Requirement	Validation	LOW	ACKNOWLEDGED	p. 69
L-20	Extra Collateral Can Be Withdrawn	Warning	LOW	ACKNOWLEDGED	p. 70
L-21	Market Initial Price Can Be Changed	MEV	LOW	ACKNOWLEDGED	p. 71
L-22	Claim Max. Length Is Not Restricted	Validation	LOW	ACKNOWLEDGED	p. 72

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-23	assertTruth Can Revert	DoS	LOW	ACKNOWLEDGED	p. 73
L-24	0 Value Transfers Are Allowed in updateCollateral()	Unexpected Behavior	LOW	RESOLVED	p. 74
L-25	assertionResolvedCallback Function Ignores assertedTruthfully	Logical Error	LOW	ACKNOWLEDGED	p. 75
I-01	Use Call Instead Of Transfer For Native Assets Transfers	Warning	INFORMATIONAL	RESOLVED	p. 79
I-02	Debug Statements In Production Code	Best Practices	INFORMATIONAL	RESOLVED	p. 80
I-03	Unused Parameters	Best Practices	INFORMATIONAL	RESOLVED	p. 81
I-04	Lack Of A Double Step Transfer Ownership Pattern	Best Practices	INFORMATIONAL	PARTIALLY RESOLVED	p. 83
I-05	Missing Events for Configuration Setter Functions	Events	INFORMATIONAL	RESOLVED	p. 84
I-06	Inconsistent Storage Slot Naming in ERC721 Implementation	Warning	INFORMATIONAL	RESOLVED	p. 86
I-07	Protocol Do Not Support Fee On Transfer Tokens	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 87
I-08	Reference To Deprecated vGas Tokens In Error Message	Warning	INFORMATIONAL	RESOLVED	p. 88
I-09	Omission of Explicit Security and Executor LZ Configurations	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 89
I-10	Absence of Excess Collateral Refund in LP Position Creation	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 90
I-11	Unused Code	Superfluous Code	INFORMATIONAL	RESOLVED	p. 92
I-12	Missing Access Control For Virtual Token	Access Control	INFORMATIONAL	RESOLVED	p. 93

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-13	Unnecessary Returned Values	Superfluous Code	INFORMATIONAL	RESOLVED	p. 94
I-14	Deadline Is Validated Twice	Superfluous Code	INFORMATIONAL	RESOLVED	p. 95
I-15	Overwritten Returned Parameters	Logical Error	INFORMATIONAL	RESOLVED	p. 96
I-16	Unused Param In _sendBalanceUpdate	Gas Optimization	INFORMATIONAL	RESOLVED	p. 97
I-17	Missing INftModule In ISapience	Informational	INFORMATIONAL	RESOLVED	p. 98
I-18	Inefficient Usage of DURATION_MULTIPLIER	Gas Optimization	INFORMATIONAL	RESOLVED	p. 99
I-19	Layer Zero Refunds Trigger ETHDeposited Event	Events	INFORMATIONAL	RESOLVED	p. 100
I-20	Avoid Using _lzSend	Gas Optimization	INFORMATIONAL	ACKNOWLEDGED	p. 101
I-21	Quote Functions Use Normalized Collateral	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 102
I-22	Closing Liquidity DoS'ed By Dust Amounts	Rounding	INFORMATIONAL	RESOLVED	p. 103
I-23	Incorrect FeeManagement Comment	Informational	INFORMATIONAL	RESOLVED	p. 105
I-24	Disputes Force Settlement Process To Be Reset	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 106
I-25	getPnl Function May Return Stale Data	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 107
I-26	Manual Settlement Can Not Set The Price To 0	Warning	INFORMATIONAL	RESOLVED	p. 108
I-27	Unused Encoder Function	Informational	INFORMATIONAL	RESOLVED	p. 109
I-28	Note About withdrawCollateral Return Value	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 110

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
REMEDIATION REVIEW					
H-01	Withdrawal Race Condition Causes Balance Desync	Logical Error	HIGH	ACKNOWLEDGED	p. 111
H-02	removeWithdrawalIntent() can be abused	Gaming	HIGH	ACKNOWLEDGED	p. 113
M-01	Dispute Flag Prevents Market Settlement Even After UMA Confirms Truth	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 115
M-02	Innaccurate tradeSlippage Parameter	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 117
M-03	UMALayerZeroBridge Only Supports A Single bondToken	Configuration	MEDIUM	ACKNOWLEDGED	p. 120
L-01	Incorrect ERC-721 Safe Mint Check Before Minting And With Wrong From Address	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 121
L-02	Redundant intent.executed Withdrawal Flag	Gas Optimization	LOW	ACKNOWLEDGED	p. 122
L-03	Dust Liquidity Decrease Can Trigger Revert	Rounding	LOW	ACKNOWLEDGED	p. 123
L-04	Gas threshold are checked after message is sent	Events	LOW	ACKNOWLEDGED	p. 124
L-05	Race Condition May Block The Settlement	DoS	LOW	ACKNOWLEDGED	p. 125
I-01	Invalid Comment About depositCollateral	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 126
I-02	Outdated Comment On Fee Collector	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 127
I-03	Note about assertedTruthfully	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 128

HIGH **H-01**

RESOLVED

Persistent Withdrawal Intent Blocks Subsequent Bond Withdrawals

CATEGORY Logical Error
LOCATION BondManagement.sol
REVIEW Main Review

DESCRIPTION

In the `BondManagement` abstract contract (inherited by `UMALayerZeroBridge`), the `executeWithdrawal` function marks a withdrawal intent as executed by setting `intent.executed = true` but does not clear the `withdrawalIntents[msg.sender][bondToken].amount` value. This leaves the `amount` field unchanged, causing any future calls to `intentToWithdrawBond` for the same `submitter` and `bondToken` to revert due to the check `require(withdrawalIntents[msg.sender][bondToken].amount == 0, "Withdrawal intent already exists");`. As a result, users with remaining bond balances after a partial withdrawal cannot initiate new intents, locking their remaining funds in the escrow indefinitely.

The relevant code in `BondManagement.executeWithdrawal` only updates the `executed` flag and subtracts from the balance, without resetting the intent:

```
function executeWithdrawal(address bondToken) external virtual nonReentrant returns
(MessagingReceipt memory) {
    // ...
    intent.executed = true; // Only this flag is set; amount remains >0
    submitterBondBalances[msg.sender][bondToken] -= intent.amount;
    // ...
}
```

This persists the non-zero amount, triggering the revert in `intentToWithdrawBond`:

```
function intentToWithdrawBond(address bondToken, uint256 amount)
external
virtual
nonReentrant
returns (MessagingReceipt memory)
{
    // ...
    require(withdrawalIntents[msg.sender][bondToken].amount == 0, "Withdrawal intent already
```

H-01 Persistent Withdrawal Intent Blocks Subsequent Bond Withdrawals

CONTINUED

```
exists"); // Fails if prior amount lingers  
// ...  
}`
```

• Exploitation scenario: (1) User deposits 100 bond tokens. (2) Creates intent to withdraw 50. (3) After delay, executes withdrawal of 50 (balance now 50, intent.amount still 50, executed=true). (4) Tries to withdraw remaining 50 → `intentToWithdrawBond` reverts on non-zero amount. (5) Funds stuck; no way to clear intent without owner intervention.

RECOMMENDATION

When `CMD_FROM_ESCROW_WITHDRAW` is executed on the market side, send confirmation back to the UMA chain which marks the intent as delivered. Then check if the intent is already delivered in `intentToWithdrawBond` and if so, allow creating a new one.

IMPORTANT: Do not allow creating a new intent if the `executed` flag of the current one is `true`. Doing so may introduce new race conditions and result in accounting mismatch.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

HIGH **H-02**

RESOLVED

Manual Settlement Yields Fractional/Innaccurate Payouts

CATEGORY Logical Error
LOCATION SettlementModule.sol
REVIEW Main Review

DESCRIPTION

The protocol provides an emergency function `SettlementModule.__manual_setSettlementPrice` that finalizes any still-unresolved market after a delay of twice the market's duration. The function retrieves the instantaneous Uniswap V3 pool price (via `slot0.sqrtPriceX96`), converts it to a decimal, clamps it to the configured `[minPriceD18, maxPriceD18]` band, and then marks the market as settled.

While this may loosely approximate a numeric outcome, for binary (Yes/No) markets it utterly destroys the design intent as these markets are meant to resolve to exactly one of two canonical values, `minPriceD18` or `maxPriceD18`, representing a definitive "NO" or "YES." By locking in whatever continuous value the pool happens to show, say, 0.73 e18, the contract transforms a winner-takes-all payoff into a proportional split, so YES-token holders receive only seventy-three per cent of their rightful collateral and NO-token holders keep the remainder. This shortfall is a pure, uncompensated loss of profit and loss (PNL) for the honest winning side.

Worse still, `slot0`'s spot price simply reflects the liquidity-weighted consensus of recent traders and liquidity providers, not the actual real-world truth of the question. It therefore advantages whichever side commanded more pool liquidity, allowing the majority sentiment to dictate a fractional "resolution" that can be entirely `false`. Honest participants can see a perfectly valid "YES" resolution diluted or reversed simply because the pool was loaded more heavily on the NO side at that moment.

This vulnerability is compounded by the broken dispute flow in `UMASettlementModule.assertionResolvedCallback`. The code ignores the `assertedTruthfully` flag returned by UMA as, any dispute, regardless of the final oracle verdict, sets `settlement.disputed = true` and permanently blocks automatic settlement. An adversary anticipating an unfavorable DVM outcome can therefore pay a single bond to lodge a dispute, stall the oracle-verified resolution, wait out the forced delay, and then invoke manual settlement. At that point, the attacker chooses the block to sample the pool price, benefiting from the majority-weighted liquidity position to reshape payouts in their favor. Not only is the binary outcome

H-02 Manual Settlement Yields Fractional/Innacurate Payouts

CONTINUED

falsified, but the rightful winners also forfeit a portion of their PNL and remain unable to withdraw until the market is forcibly closed.

In summary, this design allows a losing party to salvage value or even profit by stalling the oracle and then hijacking the emergency mechanism, fundamentally breaking the economic guarantees and trust model of binary markets.

— RECOMMENDATION

Disable the manual-settlement escape hatch for any market defining a `claimStatementNo` (i.e., binary or categorical markets). If an emergency fallback is deemed necessary, require the caller to supply an explicit boolean, which the contract then maps internally to `minPriceD18` or `maxPriceD18`, preserving the discrete nature of payouts. Additionally, update `assertionResolvedCallback` to honor the `assertedTruthfully` flag: when UMA's DVM confirms the original assertion, the settlement price must be finalized immediately regardless of prior disputes, and only a `false` verdict should leave the market unsettled. These changes reinstate the oracle as the sole source of truth and eliminate the profit opportunity created by the current manual path.

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/631>

HIGH **H-03**

RESOLVED

Zero-Price Settlement Can Create Immediate Bad Debt in Binary Markets

CATEGORY Logical Error
LOCATION Market.sol
REVIEW Main Review

DESCRIPTION

Binary (YES/NO) markets are implemented as a Uniswap-V3 pool whose lower tick corresponds to the "NO" payoff and whose **upper tick** corresponds to the "YES" payoff.

Before a trade or an LP action is accepted, Sapience computes the worst-case loss each position could suffer by assuming price will move to the pool's configured `minPriceD18` (derived from `lowTick`) or `maxPriceD18` and then requires collateral large enough to cover that loss.

At settlement, governance (or the oracle) submits the final outcome through `setSettlementPriceInRange`. That function clamps any non-zero price into the configured `[minPriceD18, maxPriceD18]` interval, but makes an exception for an exact 0:

```
function setSettlementPriceInRange(Data storage self, uint256 settlementPriceD18) internal
returns (uint256) {
    // Special case: allow exact 0 for complete loss scenarios (e.g., "No" outcome in yes/no
    markets)
    if (settlementPriceD18 == 0) {
        self.settlementPriceD18 = 0; // <-----
    } else if (settlementPriceD18 > self.maxPriceD18) {
        self.settlementPriceD18 = self.maxPriceD18;
    } else if (settlementPriceD18 < self.minPriceD18) {
        self.settlementPriceD18 = self.minPriceD18;
    } else {
        self.settlementPriceD18 = settlementPriceD18;
    }
    self.settled = true;
    return self.settlementPriceD18;
}
```

H-03 Zero-Price Settlement Can Create Immediate Bad Debt in Binary Markets

CONTINUED

Consequently, if market creators pick a realistic "NO" tick (e.g. $-92200 \Rightarrow \approx 0.0001$ \$) to improve UX, the protocol:

1. Collects collateral assuming the worst price is 0.0001 \$.
2. Allows settlement at exactly 0 \$.
3. Realizes losses up to 0.0001 \$ per virtual base unit beyond what was collateralized.

Any long-YES trader or full-range LP is now under-collateralized. Because repayments inside `Position.settle()` are not optional, negative balances are socialised, creating protocol-wide bad debt.

The last users who call `settlePosition` will not receive their full payout as the call will revert with an underflow error.

— RECOMMENDATION

Consider removing the zero-price special case. On the other hand, in `createMarket`, when the market is a binary market, require `lowTick` to be so close to the Uniswap minimum (≤ -887260) that `minPriceD18` is economically indistinguishable from 0 (e.g. < 1 wei).

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/6883a4a737f931fbfd01997be56ab844>

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/632>

HIGH **H-04**

RESOLVED

depositBond Function Can Be Abused To Drain Bridge ETH

CATEGORY Validation
LOCATION BondManagement.sol
REVIEW Main Review

DESCRIPTION

The `BondManagement.depositBond` function inherited by `UMALayerZeroBridge` is public and lacks sufficient validation on the `bondToken` parameter, allowing any user to deposit arbitrary ERC20 tokens. While it includes basic checks for non-zero address and amount, it performs a `safeTransferFrom` and then triggers a LayerZero cross-chain message via `_sendBalanceUpdate` to sync the remote balance on the Market-side bridge. This LZ send is paid from the contract's ETH reserves (`fee.nativeFee`, typically 0.0002-0.005 ETH per message based on LZ docs and gas costs).

The core issue is that the contract bears the LZ native fee cost for every deposit, regardless of the token's validity or usability for UMA assertions. Since the function is external, attackers can repeatedly call it with junk tokens (by deploying worthless ERC20s), forcing the bridge to pay fees each time. This drains the operational ETH (deposited via `depositETH` for relaying), potentially halting critical functions like assertions or callbacks when ETH drops below required fees.

Code snippet showing the fee payment:

```
MessagingReceipt memory receipt = _sendBalanceUpdate(
    _getDepositCommandType(), msg.sender, bondToken, submitterBondBalances[msg.sender][bondToken],
    amount
);
// ... (updates balance)
```

Inside `_sendBalanceUpdate` → `_sendLayerZeroMessageWithQuote`:

```
fee = _quote(...);
receipt = this._sendMessageWithETH{value: fee.nativeFee}(...); // Contract pays ETH
```

RECOMMENDATION

H-04 depositBond Function Can Be Abused To Drain Bridge ETH

CONTINUED

Add token whitelisting by querying UMA's `CollateralWhitelist.isOnWhitelist(bondToken)` before transfer, and enforce a minimum deposit amount covering the LZ fee quote (e.g., `amount >= fee.nativeFee` in bondToken value, or simply require user `msg.value >= fee.nativeFee`).

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

HIGH **H-05**

ACKNOWLEDGED

Total Loss For LPs In Binary Markets

CATEGORY Logical Error
LOCATION Global
REVIEW Main Review

— DESCRIPTION

For binary prediction markets, one of the tokens will always expire worthless (i.e., either "Yes" or "No" will have a price of 0 at settlement). This introduces a structural flaw: towards the end of the epoch, once the likely outcome becomes clear, a trader can simply swap heavily into the winning token, dumping the soon-to-be-worthless token onto the LPs.

This means LPs bear the full brunt of the market's resolution and are left with only the losing side. Since these tokens have no value post-settlement, LPs effectively suffer a complete loss every time a binary market ends. This is especially problematic for low-liquidity markets where price manipulation becomes easier and the cost to arbitrage late in the epoch is low.

As a result, LPs have no incentive to provide liquidity to binary markets under this design, as the risk-reward profile is heavily skewed against them.

— RECOMMENDATION

Consider capping the winning side's payout to the total collateral contributed by the losing side. This would ensure that LPs do not get abused. However, the implementation may be more complex, requiring proportional distribution logic at settlement.

Alternatively, reconsider the use of AMM design for binary outcome markets. As described in this Paradigm [article](https://www.paradigm.xyz/2024/11/pm-amm?), AMMs are generally unsuitable for this use case due to asymmetric payoff structures and end-of-market exploits.

PROOF OF CONCEPT <https://gist.github.com/giraffe0x/608e7b30cd9c6c307b09b2e1cee687a3>

— RESOLUTION

Status: Acknowledged.

HIGH **H-06**

RESOLVED

Insufficient Slippage When Closing LPs

CATEGORY MEV

LOCATION LiquidityModule.sol: 268-269

REVIEW Main Review

DESCRIPTION

Users can pass a desired `liquiditySlippage` parameter to `LiquidityModule.closeLiquidityPosition()` which should serve as a protection for the LPs during price movements. The `LiquidityModule` will calculate a minimum amount of tokens that should be received after the liquidity is removed based on that parameter.

```
amount0Min: stack.previousAmount0.mulDecimal(DecimalMath.UNIT - params.liquiditySlippage),
amount1Min: stack.previousAmount1.mulDecimal(DecimalMath.UNIT - params.liquiditySlippage),
```

However, `stack.previousAmount0` and `stack.previousAmount1` are the amounts currently held by the liquidity position. Any price movement that happened before that will impact these amounts, and when all liquidity is removed the full amounts will be received. This makes the `liquiditySlippage` parameter meaningless.

An attacker can sandwich the liquidity removal transaction to change the price, the LP will then receive their tokens at a worse ratio causing loss for them and profit for the attacker when they complete the sandwich.

If the position in which the malicious trader is going is short, even the `tradeSlippage` parameter will be meaningless, since it is applied to `depositedCollateralAmount`, but short positions cause increase in the `base` token which is not added to that collateral.

RECOMMENDATION

Use absolute values for minimum tokens received like you do in `decreaseLiquidityPosition()`

PROOF OF CONCEPT <https://github.com/GuardianOrg/sapience-team2/pull/1/files>

H-06 Insufficient Slippage When Closing LPs

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/632>

MEDIUM **M-01**

ACKNOWLEDGED

Protocol Is Vulnerable To Just In Time Liquidity

CATEGORY Warning
LOCATION LiquidityModule.sol
REVIEW Main Review

DESCRIPTION

The Sapience protocol, which leverages Uniswap V3 pools for liquidity provision and trading, is susceptible to Just-In-Time (JIT) liquidity attacks where users can temporarily provide concentrated liquidity in a narrow price range immediately before large swaps are executed, thereby capturing a significant portion (up to 99%) of the swap fees generated by that trade before removing the liquidity. This strategy exploits the pro-rata fee distribution in Uniswap V3, where fees are allocated based on liquidity share during the swap, allowing attackers to dominate the active tick range and self-capture fees from their own trades, reducing their net trading costs while diluting fee earnings for passive liquidity providers.

In the provided test scenario (`test_Just_In_Time_Liquidity`), LP2 demonstrates this by adding 1000000e18 collateral in a narrow range (-7000 to -6800, around ~0.5\$), followed by a large long trade (+50e18) and short trade (-50e18) that cross the range and generates fees, after which LP2 closes the position to collect ~0.5e18 in profit, resulting in a net gain.

While general JIT carries risks like impermanent loss from price slippage in one-sided swaps, this specific test highlights a low-risk variant where two opposite trades of equal size (+50e18 long followed by -50e18 short) occur within the same block, balancing the price back to the starting point (~0.5000\$ pre to ~0.5000\$ post), minimizing impermanent loss to near-zero as the pool rebalances symmetrically without net drift, allowing the attacker to earn fees from both swaps (total ~0.5e18 at 1% fee on ~50e18 inputs each) with negligible exposure.

RECOMMENDATION

M-01 Protocol Is Vulnerable To Just In Time Liquidity

CONTINUED

To mitigate this issue, implement anti-JIT mechanisms in the `LiquidityModule` by adding a minimum liquidity holding period (e.g., via timestamp checks in `decrease` / `closeLiquidityPosition`, requiring at least 1-5 blocks or a time delay before removal), which would force attackers to bear impermanent loss risk over multiple blocks and deter flashloan-based execution.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/4315a7d8fcac7e0359887a792eb22250>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

market.sqrtPriceMaxX96 Is Increased By Tick Spacing

CATEGORY Warning
LOCATION Market.sol
REVIEW Main Review

DESCRIPTION

In the Sapience protocol, binary markets, designed to resolve strictly at 0 (No) or 1 (Yes), require participants on the losing side to forfeit their entire stake, with winners claiming the full pot minus fees. However, testing revealed that when markets resolve to "Yes," "No" position holders (losers) can unexpectedly withdraw a portion of their collateral, violating the all-or-nothing expectation. This stems from an off-by-one error in how the market's maximum price is initialized.

During market creation, the upper bound `sqrtPriceMaxX96` is set to the sqrt ratio at `baseAssetMaxPriceTick + tickSpacing` (e.g., tick 0 + 200 = 200 for 1% fee tier), resulting in a `maxPriceD18` slightly above $1e18$ (approximately $1.0202e18$ for spacing=200). Consequently, collateral requirements in `getCollateralRequirementsForTrade` use this inflated `maxPriceD18` when valuing borrowed `vBase` for short ("No") positions at the upper extreme, forcing them to post excess collateral (about 2% more) to cover a hypothetical debt value greater than 1. At settlement, with the price capped at exactly 1, this overage is returned to losers as unneeded collateral, allowing them to reclaim funds that should have been fully transferred to winners.

This happens because binary markets intend an exact upper price of 1, but the `+spacing` adjustment pushes it higher without corresponding adjustments. This inefficiently locks extra capital for "No" voters throughout the market's life, potentially deterring participation.

RECOMMENDATION

To ensure binary markets enforce exact bounds of 0 to 1, adjust the initialization of `baseAssetMaxPriceTick` during market creation to be one `tickSpacing` below 0 (e.g., -200 for a 1% fee tier), so that `sqrtPriceMaxX96` aligns precisely with a price of 1 after adding the spacing offset.

M-02 market.sqrtPriceMaxX96 Is Increased By Tick Spacing

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Unreduced Debt Following Partial Liquidity Withdrawals Amplifies Impermanent Loss Exposure

CATEGORY Warning
LOCATION Market.sol
REVIEW Main Review

DESCRIPTION

In the `LiquidityModule`, the handling of partial liquidity decreases through the `decreaseLiquidityPosition` function introduces a significant asymmetry compared to full closures, resulting in liquidity providers (LPs) maintaining full borrowed debt levels even after reducing their active liquidity exposure. This design leads to an over-leveraged position where the remaining liquidity bears the burden of the entire initial debt, thereby magnifying impermanent loss effects during subsequent market movements. To understand this, consider the core mechanics: when an LP adds liquidity, they borrow virtual base (`vBase`) and quote (`vQuote`) tokens proportional to the provided liquidity amount and current price, backing this debt with collateral. The debt ensures the protocol's solvency by representing the tokens minted and added to the Uniswap V3 pool.

On a full closure via `_closeLiquidityPosition`, the function explicitly collects all principal and accrued fees from the position manager, nets them against the full debt (reducing `borrowedVBase` and `borrowedVQuote` to zero where possible), rebalances any excess into virtual amounts or collateral and returns the net value, effectively derisking the entire position at the closure price.

In contrast, a partial decrease burns the specified liquidity delta, accrues prorated fees and principal to `tokensOwed0` and `tokensOwed1` (fixed token amounts no longer subject to IL), and updates the position's collateral requirements using these owed as credit offsets against the unchanged full debt. However, without collecting or prorating the debt downward, the position retains its original borrow levels, now supported by reduced active liquidity plus the fixed owed credits. This mismatch becomes problematic in volatile markets: the remaining liquidity continues to rebalance (suffering IL as price shifts concentrate holdings in the lower-value token), but the full debt amplifies any valuation shortfalls at extremes during settlement. For instance, when the market price drops after the partial liquidity withdrawal the remaining active part of the position ends up holding more of the `vBase` token (which is losing value in the drop). The part already withdrawn is locked

M-03 Unreduced Debt Following Partial Liquidity Withdrawals Amplifies Impermanent Loss Exposure

CONTINUED

as fixed amounts (owed to you) from when the price was higher, so it might have more stable `vQuote` tokens, but at settlement, the system looks at the whole position: the weakened remaining part (now worth less due to the drop) has to cover the original debt. This causes bigger losses than if the debt had been reduced proportionally when the liquidity was withdrawn.

The POC attached illustrates this issue clearly: in the "exposed" path, where the LP performs a 50% partial liquidity withdrawal, the final profit and loss (PNL) at settlement (with the market price at \$1 after a temporary price drop) comes out to about $-8e18$. In contrast, the "control" path, where the LP fully closes the position (securing value at a higher price) and then re-adds half the liquidity, results in a PNL of $+1.5e18$. This gap does not occur because of the impermanent loss on the uncollected owed amounts (which are locked as fixed tokens and immune to such loss), but rather from over-leverage: the full original debt must still be covered by the reduced liquidity, which has been weakened by impermanent loss during the price drop, leading to larger overall losses. This unfairly burdens the LP who aimed to lower their risk gradually. Although the protocol's cautious collateral checks at the market's minimum and maximum price points avoid complete undercollateralization or forced sales, it still creates unexpected drawbacks that might deter LPs from adjusting positions dynamically in volatile markets, whether numeric or binary. If LPs make multiple partial withdrawals without collecting in between, the buildup of unresolved debt could worsen these problems over time, yet there is no direct way to exploit this for quick gains or cause system-wide failure, since all debt is fully resolved at settlement.

— RECOMMENDATION

Consider documenting this risk so liquidity providers are aware of it. On the other hand, consider also addressing the debt asymmetry by refactoring the `decreaseLiquidityPosition` function to prorate `borrowedVBase` and `borrowedVQuote` downward by the liquidity reduction. This will also require more changes to differentiate between the actual accrued fees and withdrawn liquidity in the underlying UniswapV3 position, when `collect` is called during position closure or settlement.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/91bcc53c238167b183bf004fd334e79a>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

RESOLVED

Dust-1-wei Debt Can Make `closeLiquidityPosition` Revert

CATEGORY Rounding
LOCATION `LiquidityModule.sol`
REVIEW Main Review

DESCRIPTION

When an LP position is closed through `LiquidityModule.closeLiquidityPosition`, the contract first:

1. Calls `INonfungiblePositionManager.decreaseLiquidity` with all the remaining liquidity.
2. Immediately collects all tokens & fees.
3. Burns the Uni-V3 NFT.
4. Reconciles the LP's virtual balances and, if every debt token has been repaid, withdraws collateral to the user.
5. Otherwise it converts the position into a Trade and invokes `_closeTradePosition` to repay whatever debt is left.

The intention of the reconciliation step is to eliminate the 1 wei rounding artefact produced by Uni-V3:

```
// _closeLiquidityPosition()
if (collectedAmount0 > 0) { // vBase side
    collectedAmount0 += 1; // try to cover the usual 1-wei shortfall
}
...
if (collectedAmount0 > position.borrowedVBase) {
    position.vBaseAmount = collectedAmount0 - position.borrowedVBase;
    position.borrowedVBase = 0;
} else {
    position.borrowedVBase = position.borrowedVBase - collectedAmount0;
}
```

In some edge-paths `collectedAmount0` is still one wei short (e.g. because no swap fees were accumulated in the last tick or precision rounding in `LiquidityAmounts`). Consequently the position exits the LP branch with:

M-04 Dust-1-wei Debt Can Make closeLiquidityPosition Revert

CONTINUED

```
borrowedVBase = 1
borrowedVQuote = 0
vBaseAmount = 0
```

`_closeTradePosition` then tries to buy back exactly one wei of `vBase` via an exact-output swap:

```
amountOut (vBase) = 1
// therefore tradedVQuote = 2 and tradeRatio = 2 / 1 = 2e18
```

Because the market's configured upper-bound price (`maxPriceD18`) is much lower ($\approx 1.02 \text{ e18}$ in the failing test), `Trade.quoteOrTrade` reverts:

```
if (tradeRatioRoundUp > market.maxPriceD18) {
  revert TradePriceOutOfBounds(tradeRatioRoundUp, market.minPriceD18, market.maxPriceD18);
}
```

The user is stuck: every close attempt reverts even though the remaining debt is only 1 wei and the position is economically safe.

— RECOMMENDATION

Rather than rejecting the transaction, treat the price-cap check as optional when the trade is merely clearing ≤ 1 wei of debt:

```
// inside Trade.quoteOrTrade, immediately before the price-bound checks
bool isDustClose =
tradedVBase == 1 && // buying back exactly 1 wei of base
tradedVQuote <= 2 && // paying at most 2 wei of quote
targetSize == 0; // final size will be flat
if (!isDustClose) {
  // normal safety guard
  if (tradeRatioD18RoundDown < market.minPriceD18 ||
  tradeRatioD18RoundUp > market.maxPriceD18) {
    revert Errors.TradePriceOutOfBounds(
tradeRatioD18RoundDown, market.minPriceD18, market.maxPriceD18
);
  }
}
// else: skip the bound-check – the protocol is only forgiving  $\leq 2$  wei of value
```

M-04 Dust-1-wei Debt Can Make closeLiquidityPosition Revert

CONTINUED

The check is bypassed only when `|borrowedVBase| == 1 wei` and `borrowedVQuote == 0;` the maximum value that can be over- or under-charged is therefore 2 wei, which is economically negligible while restoring UX.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/b620192a003f0638e7ddf70b1d7489b6>

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/632>

MEDIUM **M-05**

RESOLVED

Race Condition in Bridged Bond Withdrawal

CATEGORY	Race Condition
LOCATION	MarketLayerZeroBridge.sol, UMLayerZeroBridge.sol
REVIEW	Main Review

DESCRIPTION

The bridged bond withdrawal process across `UMLayerZeroBridge` (UMA side) and `MarketLayerZeroBridge` (Market side) is vulnerable to a race condition due to asynchronous LayerZero message delivery, causing temporary desynchronization between escrowed balances and remote views and a permanent loss of funds for the user affected.

When a withdrawal intent is created on the UMA side, it validates sufficient balance and no existing intent, creates a local `withdrawalIntents` entry, and sends a delayed LZ message to update the Market side's `remoteSubmitterWithdrawalIntent`. However, if the message is delayed, an intervening assertion on the Market side can deduct from `remoteSubmitterBalances` before the intent reserves it, leaving the UMA escrow sufficient for execution but causing the subsequent withdrawal LZ message to revert on the Market side due to insufficient remote balance. This keeps the UMA-side intent marked as executed (funds transferred to user), but the Market-side `remoteIntent` remains set and balance undeducted, blocking new intents for that user/token pair (via `require(withdrawalIntents[msg.sender][bondToken].amount == 0)`) without a reset mechanism.

No cancel function exists, trapping the remaining user's escrowed `bondTokens`.

Proof of Concept

A scenario illustrating this with an initial 1200 USDC balance for the user Bob, 800 USDC intent and 500 USDC assertion bond:

- T=0: Bob calls `intentToWithdrawBond(USDC, 800)` on UMA side, validating balance and no prior intent, sending a LayerZero message (that will be delayed), then creating local intent. UMA balance: 1200 USDC.

```
// UMA Chain - UMLayerZeroBridge contract
withdrawalIntents[msg.sender][bondToken] = BridgeTypes.WithdrawalIntent({amount: amount,
timestamp: block.timestamp, executed: false});
withdrawalIntents[Bob][USDC] = BridgeTypes.WithdrawalIntent({amount: 800, timestamp:
block.timestamp, executed: false});
```

M-05 Race Condition in Bridged Bond Withdrawal

CONTINUED

• T=1: MarketGroup calls `MarketLayerZeroBridge.forwardAssertTruth(marketGroup, marketId, claim, asserter, liveness, currency, bond)`, where `asserter` is Bob.

```
// Market Chain - MarketLayerZeroBridge contract
remoteSubmitterBalances[asserter][currency] -= bond;
remoteSubmitterBalances[Bob][USDC] = 1200 - 500 = 700;
```

• T=2 min: Assert processes on UMA side, deducting 500 from escrow (balance now 700), escrowing in UMA.
 • T=6 min: Withdrawal intent arrives on Market side, setting `remoteIntent=800` (view: balance=700, intent=800).

```
// Market Chain - MarketLayerZeroBridge contract - _lzReceive
remoteSubmitterWithdrawalIntent[submitter][bondToken] = deltaAmount;
remoteSubmitterWithdrawalIntent[Bob][USDC] = 800;
```

• T=1441 min (1 day later = withdrawal delay): Bob calls `executeWithdrawal` successfully on the UMA Chain - `UMALayerZeroBridge` contract. When this LayerZero message arrives to the Market Chain - `MarketLayerZeroBridge` contract it reverts as $(700 < 800)$:

```
// Market Chain - MarketLayerZeroBridge contract
(address submitter, address bondToken, uint256 deltaAmount) =
data.decodeFromBalanceUpdate();
remoteSubmitterBalances[submitter][bondToken] -= deltaAmount;
remoteSubmitterBalances[Bob][USDC] = 1200 - 500 = 700 -= 800; // Reverts here
```

Bob, at this point, loses access to escrowed bonds indefinitely.

— RECOMMENDATION

Implement a user-callable `cancelWithdrawalIntent` function on the UMA side to clear stuck intents (set amount=0, executed=false) after failed execution, sending an LZ message to reset Market-side `remoteIntent` to 0, conditional on no active assertions referencing the bond.

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

MEDIUM **M-06**

ACKNOWLEDGED

Risk Of assertionId Collision In UMAOracle

CATEGORY	Logical Error
LOCATION	UMASettlementModule.sol: 139
REVIEW	Main Review

DESCRIPTION

In UMA's `OptimisticOracleV3`, the `assertTruth` function assigns each assertion a unique ID based on the hash of several input parameters:

```
_getId(claim, bond, time, liveness, currency, callbackRecipient, escalationManager, identifier)
```

However, `UMASettlementModule` submits highly standardized data to `assertTruth`, which can result in ID collisions when multiple assertions are submitted within the same block.

In particular, the call from `submitSettlementPrice` uses the following:

```
market.assertionId = optimisticOracleV3.assertTruth(
  claim,
  params.asserter,
  address(this),
  address(0),
  marketGroup.marketParams.assertionLiveness,
  IERC20(marketGroup.marketParams.bondCurrency),
  marketGroup.marketParams.bondAmount,
  optimisticOracleV3.defaultIdentifier(),
  bytes32(0)
);
```

These parameters are often identical across markets. The one parameter expected to introduce uniqueness is `claim`. However, `getClaim` currently only returns claim statements (which might be simple Yes/No string values). Thus, when two or more assertions with identical data are submitted in the same block, they produce the same assertion ID causing collision.

For bridged settlements, this issue is particularly concerning because the unexpected revert will occur at the UMA chain, while the market chain assumes the assertion was successful, preventing a re-submission.

M-06 Risk Of assertionId Collision In UMAOracle

CONTINUED

— RECOMMENDATION

Include a unique identifier such as `marketId` within the `claim` string to prevent collisions between assertions tied to different markets.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

Arbitrary Asserter Allows Bond Theft

CATEGORY	Logical Error
LOCATION	UMASettlementModule.sol, MarketLayerZeroBridge.sol
REVIEW	Main Review

DESCRIPTION

In the `submitSettlementPrice` function, if the market group is `bridgedSettlement`, the bond amount is deducted from the assenter's balance:

```
remoteSubmitterBalances[asserter][currency] -= bond;
```

However, there are no validations to ensure that the assenter is associated with the correct market group, market ID, or even the caller. This omission enables a serious attack where an adversary can drain the bond balance of honest users by exploiting the UMA assertion-dispute mechanism.

Example attack scenario:

1. Setup: Alice (attacker) is the owner of a market group using `MarketLayerZeroBridge` for settlement.
2. Target: Bob (honest user) deposits a bond, intending to participate in assertions for a different market group but also using the same bridge.
3. Exploit: Alice calls `submitSettlementPrice`, setting `asserter = Bob` -- this deducts Bob's bond balance even though he is unrelated to this assertion.
4. Profit: Alice asserts a deliberately false claim, immediately disputes her own assertion, and ultimately wins the dispute, gaining Bob's bond.

This attack is made possible because in the bridged flow:

- The bond is tied to the assenter, not the caller.
- There is no guarantee that the assenter is the rightful owner or participant in the context of the market group or market ID.

In contrast, the non-bridged flow correctly ensures that the caller is always the one who posts the bond, preventing such abuse.

RECOMMENDATION

M-07 Arbitrary Asserter Allows Bond Theft

CONTINUED

The bond deduction logic in `submitSettlementPrice` should be redesigned to always charge the caller, not an arbitrary asserter. The caller's bond can then be transferred cross-chain to the UMA bridge and tied to their assertion.

An alternate solution could be to ensure that bond deposits are mapped to market group and market id. Then during `submitSettlementPrice` validate that the asserter had deposited a bond for that specific market group and market id.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

RESOLVED

Incorrect Market Params Used

CATEGORY	Logical Error
LOCATION	UMASettlementModule.sol: 22
REVIEW	Main Review

DESCRIPTION

The current implementation updates `epoch` to `market` and `market` to `marketGroup`. This `marketGroup` is basically the `Sapience` diamond contract, that will be initialized with certain `marketParams`.

When a `market` is created, the `marketGroup.marketParams` is cached inside the `market` struct in order to prevent any changes to the `marketGroup` by the owner. However, the market's cached params are not always used in the current implementation and instead use the `marketGroup.marketParams` which can change at any time.

This opens up different issues when `marketGroup.marketParams` are updated like:

- `bondCurrency` changes, user that already deposited a different token will need to withdraw and deposit the new bond currency
- when creating liquidity positions, if the `feeRate` is updated, it will try to mint tokens from a completely different UniswapV3 pool
- when there is a bridged settlement, `marketParams.optimisticOracleV3` is the `MarketLayerZeroBridge`. If this contract is updated, users that have previously deposited or initiated a withdrawal will be DoS'ed as the new contract won't have these values.

RECOMMENDATION

Consider using the cached `market.marketParams` values instead of the `marketGroup.marketParams`.

Keep in mind there is an edge case for the `bondAmount` in non-bridge settlements. If OOV3 increases the minimum bond and the market cached a lower value during creation, settlements will revert as it will be forced to use that cached value instead. Even if the owner increases the `bondAmount`, this will not affect the cached values.

RESOLUTION

Status: Resolved.

M-08 Incorrect Market Params Used

CONTINUED

<https://github.com/sapiencexyz/sapience/pull/632>

MEDIUM **M-09**

ACKNOWLEDGED

Slippage Should Be Applied For Trading Positions

CATEGORY MEV
LOCATION Trade.sol
REVIEW Main Review

DESCRIPTION

When traders create or modify their positions, they trade virtual tokens in the Uniswap pool via `Trade.sol`. There are no slippage parameters provided there - `sqrtPriceLimitX96` and `amountOutMinimum` are 0, `amountInMaximum` is `type(uint256).max`.

According to the comment provided next to these slippage parameters, that's okay since the more tokens a position has borrowed, the more collateral it will require and the user has control over how much collateral can be pulled from their account via the `maxCollateral` and `deltaCollateralLimit` fields.

However, when the price is being manipulated during a sandwich, the `PnL` of the position will affect the current `depositedCollateralAmount` of the position, therefore the delta value as well.

For example:

- Alice opens a X long and deposits Y collateral @ Z\$
- Later, Alice wants to increase her position, she quotes Sapience and sees the needed collateral is $Y + D$, so she sets the `deltaCollateralLimit` = $1.2 * D$ (20% slippage).
- Bob frontruns her transaction with his long which will move the price up
- Alice will buy the base tokens at a much higher price, but because the price has increased, she will also accumulate `PnL` which will increase `depositedCollateral` amount and will result in a lower `deltaCollateral`
- If the `deltaCollateral` is in the limits of $D * 1.2$, the transaction will pass through, Alice will buy the tokens at unfavorable price and Bob will frontrun her for profit.

RECOMMENDATION

Allow the traders to specify `amountInMaximum` and `amountOutMinimum`

PROOF OF CONCEPT <https://github.com/GuardianOrg/sapience-team2/pull/3>

M-09 Slippage Should Be Applied For Trading Positions

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-10**

RESOLVED

Repeated Liquidity Decrease Reverts Due Rounding

CATEGORY Logical Error
LOCATION LiquidityModule.sol
REVIEW Main Review

DESCRIPTION

Liquidity providers (LPs) calling `decreaseLiquidityPosition` multiple times experience a revert due to the required collateral being `1 wei` higher than deposited.

The root cause lies in how debt amounts stay constant when partially decreasing liquidity, and how additional collateral cannot be added if there is shortfall after decreasing liquidity.

Example scenario:

1. Alice adds liquidity in the range [-200, 200], borrows 100 base and quote tokens
2. Price moves to tick 300. Alice's position is out of range (only Quote tokens remains)
3. On the first `decreaseLiquidityPosition`, she reduces her position by 10%. Though out of range, she receives both base and quote tokens from earned fees. Her credit is calculated from the remaining liquidity converted to `quote` plus `tokensOwed1`. Her debt is the borrowed base and quote tokens, less any amounts offset by tokens owed. Excess collateral is refunded to Alice
4. On the next `decreaseLiquidityPosition` call, she again removes 10% liquidity. She receives no base tokens due to being out of range, and any `tokensOwed1` returned by Uniswap is rounded down
5. This causes her credit to be `1 wei` less than the previous round, but her debt remains constant
6. Because the deposited collateral has already been refunded from the first call, it is now `1 wei` short and the transaction reverts

RECOMMENDATION

When `decreaseLiquidityPosition` is called and the deposited collateral is insufficient by a small amount, allow the LP to top up the deficit by transferring the shortfall into the Sapience contract. This prevents unnecessary reverts caused by rounding issues.

M-10 Repeated Liquidity Decrease Reverts Due Rounding

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/sapience-team1/pull/3/files#diff-54cfc5548d61fddc84443d1aad92bb107c7fdda9d27eaa99989b93d605d64570>

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/632>

MEDIUM **M-11**

ACKNOWLEDGED

Market Settlement May Be Blocked

CATEGORY Unexpected Behavior
LOCATION SettlementModule.sol: 73-102
REVIEW Main Review

DESCRIPTION

When `UMASettlementModule` handles crosschain requests, it sets `market.assertionId = bridge.forwardAssertTruth()`. Then it doesn't allow making any new assertions until that id is cleared. The problem is that if the LayerZero message initiated from `forwardAssertTruth()` reverts on the destination chain, there will be no way to clear the `assertionId`. This means the market is stuck and cannot be settled, therefore nobody can close their positions and withdraw their collateral. The `__manual_setSettlementPrice()` function in `SettlementModule` works only for the last market in the group.

Possible reasons for revert are:

- the bond amount sent to the oracle is below the minimum (there is no validation for the crosschain case)
- the used bond token has been removed from the whitelisted collaterals before the destination transaction arrived

RECOMMENDATION

Modify `__manual_setSettlementPrice()` in such a way that it accepts a market id to settle. This will save the markets from the described scenario.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-12**

RESOLVED

Manual Settlement May Be Manipulated

CATEGORY MEV
LOCATION SettlementModule.sol: 91-92
REVIEW Main Review

DESCRIPTION

The manual settlement feature allows anyone to settle the market with whatever the current price of the underlying pool is. This action can be executed if the market hasn't been settled after its duration has passed twice. While it seems safe to use the pool price because nobody besides the Sapience diamond proxy holds virtual tokens, it's still possible for a manipulation to happen if certain conditions are met. For example, if the legitimate price ended up at the bounds of a given liquidity position, i.e one of the tokens of that position has been fully depleted, a malicious user can execute empty swaps. These swaps allow them to move the price to the point where the next liquidity position starts. In the end, the market will use the manipulated price and the `settlePosition` calculations will credit users incorrectly.

RECOMMENDATION

One possible solution is to globally keep track of the latest pool price after each swap by using the `tradeRatio`. Then you can use that price in the manual settlement flow.

PROOF OF CONCEPT <https://github.com/GuardianOrg/sapience-team2/pull/2>

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/631>

MEDIUM **M-13**

ACKNOWLEDGED

Race Condition May Block The Settlement

CATEGORY	Race Condition
LOCATION	MarketLayerZeroBridge.sol, UMLayerZeroBridge.sol
REVIEW	Main Review

— DESCRIPTION

The following possibility for a race condition blocking a market from being settled exists:

- Alice deposits a bond of 1000 tokens
- Her transaction is executed on the market side and her balance is set to 1000
- She initiates a withdrawal for 600 tokens
- Before her intent is delivered on the market side, an assertion with a 500 bond is created
- The intent transaction is delivered and her intent value becomes 600
- The asserting transaction is being delayed for 1 day (for example, DVN/executor issues, etc...)
- Since 1 day passed, Alice can call `executeWithdrawal()`, which will deduct her balance to 400
- When the asserting transaction is finally delivered, it will try deducting 500 from Alice's balance, which will cause a revert.

If Alice never deposits bond again, the claim can never be asserted and since the `assertionId` was already recorded in `UMASettlementModule`, this market can now never be resolved.

— RECOMMENDATION

This scenario should be very rare since in most cases transactions are expected to be delivered in less than a day. Because of that it's best to fix the other issue with `__manual_setSettlementPrice()` to allow settling any market.

— RESOLUTION

Status: Acknowledged.

<https://github.com/sapiencexyz/sapience/pull/631>

MEDIUM **M-14**

RESOLVED

FeeCollectorNFT Enables Under-collateralized LP Actions

CATEGORY Warning
LOCATION FeeCollectorNFT.sol
REVIEW Main Review

DESCRIPTION

The `FeeCollectorNFT` is intended to mark privileged accounts, but its current integration gives holders two sweeping powers: they can supply or remove liquidity without meeting the `requiredCollateral` threshold and they are the only addresses permitted to inject collateral into an existing LP position. The first capability is a direct solvency hazard. A malicious or compromised holder can mint an LP position with negligible collateral, continuously extract fees and then abandon the position, leaving an uncovered shortfall that must be absorbed by honest users. Because minting the NFT is governed solely by the contract owner, this privilege introduces a centralization vector and bypasses the protocol's invariant that every position remains fully collateralized at all times.

RECOMMENDATION

Eliminate every special privilege attached to `isFeeCollector` except the ability to deposit additional collateral into positions. In practice this means removing the collateral-bypass branch in `LiquidityModule.updateValidLp` and any other code paths that waive minimum-collateral checks, while preserving `depositCollateral` so that trusted parties can still top-up positions and avert potential bad debt.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/632>

MEDIUM **M-15**

ACKNOWLEDGED

Market Creations DoS'ed By Existing Pool

CATEGORY DoS
LOCATION Market.sol: 143
REVIEW Main Review

— DESCRIPTION

When a market is created, a Uniswap V3 pool is deployed with the base and quote tokens. These tokens are deployed with a deterministic approach, using a salt and CREATE2.

Once the market creation transaction is pending, any user can frontrun the owner and create the Uniswap V3 pool with the deterministic tokens and fee tier. This is possible as the Uniswap V3 Factory does not validate if the tokens exist.

— RECOMMENDATION

Consider validating if the pair exist in the Uniswap V3 Factory. However, if it does exist, revert only if the pool has not been initialized yet.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Incompatibility With Non Standard ERC20 Tokens

CATEGORY Best Practices

LOCATION UMLayerZeroBridge.sol, UMASettlementModule.sol

REVIEW Main Review

DESCRIPTION

The Sapience protocol's token approval mechanisms, particularly in contracts like `UMLayerZeroBridge` and potentially others interacting with ERC20 tokens such as bond currencies, rely on the standard `IERC20` `approve` function without safeguards for non-compliant tokens. This implementation assumes that `approve` will return a boolean value indicating success or failure, as specified in the ERC20 standard; however, certain widely used tokens like USDT (Tether) on Ethereum do not adhere to this, either returning no value or reverting under specific conditions, such as when resetting a non-zero allowance without first setting it to zero. As a result, calls to approve can fail silently or revert unexpectedly when used with these tokens, leading to integration failures, denied operations like bond deposits or assertions and potential denial-of-service scenarios where users cannot interact with the protocol using affected tokens. This incompatibility is exacerbated in cross-chain or bridge contexts, where token approvals are critical for escrow and transfer operations or preventing settlements if non-standard tokens are used as bond currencies.

RECOMMENDATION

To resolve this issue, replace direct calls to `approve` with `SafeERC20`'s `safeApprove` or, preferably, `forceApprove` function, which handles non-compliant tokens by first resetting the allowance to zero if necessary before setting the new value, ensuring compatibility without relying on return values. Update all relevant contracts, such as `UMLayerZeroBridge`, to use `forceApprove` for bond token approvals during assertion handling or escrow deposits.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

LOW **L-02**

ACKNOWLEDGED

Missing Price Checks For Binary Markets Settlement

CATEGORY	Validation
LOCATION	UMASettlementModule.sol, SettlementModule.sol
REVIEW	Main Review

DESCRIPTION

The Sapience protocol's settlement process for binary markets, which are identified by a non-empty `claimStatementNo` and designed to enforce absolute outcomes where winners receive the full opposing stake minus fees while losers get nothing, permits submissions of intermediate prices between 0 and 1 despite UMA verifying only the binarized claim statement. In the `submitSettlementPrice` function, a `sqrtPriceX96` that is converted to `decimalPrice` is provided without checking if it exactly matches the market's binary extremes (`minPriceD18 ≈ 0` or `maxPriceD18 ≈ 1`).

This means that an erroneous submission (e.g., `sqrtPriceX96` yielding 0.6 instead of 1 if yes or 0 if no) could dilute rewards if not disputed, as UMA confirms the outcome category but not the precise value.

On the other hand, the manual fallback in `__manual_setSettlementPrice` exacerbates this by directly using the pool's current `slot0` price, which may land between 0 and 1 in unresolved or manipulated end-states, applying an unintended partial settlement without binary enforcement.

RECOMMENDATION

Enhance binary market handling by adding validation in `submitSettlementPrice`: for markets with `claimStatementNo`, require the submitted `decimalPrice` to be exactly `minPriceD18` (for No) or `maxPriceD18` (for Yes), reverting on intermediates and mapping the boolean choice to the precise bound prices internally while generating the corresponding claim statement. For `__manual_setSettlementPrice` of binary markets, replace the `slot0`-derived price with a binary determination, e.g., if `current price > (min + max)/2`, set to `maxPriceD18` (Yes); else `minPriceD18` (No), to ensure strict outcomes even in manual overrides.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Missing Multiple UMA Basic Checks Upon Settlement

CATEGORY	Validation
LOCATION	UMASettlementModule.sol, MarketLayerZeroBridge.sol
REVIEW	Main Review

DESCRIPTION

The Sapience protocol's bridged settlement mechanism, which forwards assertions to UMA's Optimistic Oracle V3 (OOV3) via the `forwardAssertTruth` function in `MarketLayerZeroBridge`, fails to enforce critical safeguards recommended by UMA for secure oracle operations. Specifically, the function accepts user-provided `bond` and `liveness` values without verifying against OOV3's minimum bond (queriable via `getMinimumBond(token)`) or ensuring the bond scales appropriately to the market's total value at risk (e.g., TVL or potential payout shifts).

As per UMA documentation, bonds should exceed the minimum to incentivize disputers, who receive half the proposer's bond on successful challenges, and should be elevated for high-value requests: "If data received from the oracle could potentially move large amounts of value, you may want to set a higher bond to make it more costly for an attacker to attempt to steal funds through a false proposal."

In Sapience's prediction markets, where resolutions can redistribute significant collateral (e.g., winner-takes-all in binaries), a low bond relative to TVL enables cheap attacks, proposers could submit false outcomes, forcing disputers to risk more capital than the reward justifies, or succeed if disputes are uneconomical.

Furthermore, the `liveness` parameter (assertion challenge window) lacks a minimum enforcement in `submitSettlementPrice` (`UMASettlementModule`), permitting arbitrarily short periods that violate UMA's guidance: "At this time, it is generally not recommended to set a challenge window shorter than two hours."

Short liveness reduces time for community scrutiny and disputes, increasing the likelihood of unchallenged falsehoods, especially in low-visibility markets where monitoring is sparse.

Without these checks, the protocol risks economic inefficiencies (e.g., proposers underpaying for risk) and security lapses, such as attackers proposing false settlements with minimal cost.

L-03 Missing Multiple UMA Basic Checks Upon Settlement

CONTINUED

— RECOMMENDATION

In `forwardAssertTruth` and `submitSettlementPrice`, implement runtime checks to enforce UMA's minimum bond by querying `oov3.getMinimumBond(bondCurrency)` and requiring the provided `bond >= minBond`, while scaling the required bond dynamically to a percentage of the market's TVL or potential value moved (e.g., 1-5% of total collateral at stake, as implied by UMA's value-at-risk guidance). Similarly, enforce a minimum liveness period (e.g., 2 hours or 7200 seconds) to align with UMA recommendations, reverting on shorter submissions.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

Opaque Reverts in Swap Functions During Insufficient Liquidity Scenarios

CATEGORY Validation
LOCATION Trade.sol
REVIEW Main Review

DESCRIPTION

The `Trade` library, responsible for executing and quoting swaps via Uniswap V3 integrations in the `swap0rQuoteTokensExactIn` and `swap0rQuoteTokensExactOut` functions, encounters silent or generic reverts when attempting to perform trades that exceed the available liquidity in the underlying pool, without emitting a specific error to indicate the root cause.

RECOMMENDATION

Wrap the Uniswap router and quoter calls in try-catch blocks within `swap0rQuoteTokensExactIn` and `swap0rQuoteTokensExactOut` to intercept low-liquidity reverts, then revert with a custom error like `Errors.InsufficientPoolLiquidity` that includes details such as the attempted amount and market ID.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/637>

LOW **L-05**

ACKNOWLEDGED

Excessive Gas Allocation for LayerZero Balance Updates

CATEGORY	Configuration
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

In the bond management operations such as depositing tokens, creating withdrawal intents and executing withdrawals, initiate a cross-chain synchronization of balances via the `_sendBalanceUpdate` function, which leverages LayerZero for message delivery. This update mechanism reuses the same `_sendLayerZeroMessageWithQuote` logic employed for assertion callbacks, incorporating executor options that allocate 1M gas for receipt on the destination chain. However, the receiving execution in the `MarketLayerZeroBridge` for these balance-related messages involves only simple payload decoding and state modifications, consuming far less resources.

While the gas limit itself does not dictate consumption, serving instead as a cap to prevent out-of-gas failures during execution, as per LayerZero's enforced options mechanism (where developers set the maximum gas the executor should allocate), requesting an excessively high limit can inflate the upfront fee quote, as LayerZero's pricing factors in the potential execution cost based on the specified cap. Essentially LayerZero uses this gas limit to calculate the fee quote: the amount you pay upfront to cover the cost of executing the message on the destination chain. The fee is based on the gas limit you request, multiplied by the gas price on the destination chain, plus other protocol fees.

Consequently, the protocol may incur elevated bridging expenses without corresponding benefits, particularly in scenarios with frequent bond activities, though actual gas used remains low and unspent portions are not refunded.

— RECOMMENDATION

Adjust the LayerZero executor `lzReceive` gas option to a lower value specifically for balance update messages to better align with observed consumption.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

UMA Bridge Owner Can Influence Settlements

CATEGORY Trust Assumptions
LOCATION UMALayerZeroBridge.sol: 60-62
REVIEW Main Review

DESCRIPTION

Market groups with `bridgedSettlement = true` use the `MarketLayerZeroBridge` contract as its `optimisticOracleV3`. Anyone can deploy such bridge to be used by market groups. Each `MarketLayerZeroBridge` is paired with an `UMALayerZeroBridge` on the UMA side. The UMA oracle notifies the UMA bridge for any resolved or disputed assertions and a message to `MarketLayerZeroBridge` is initiated. Once executed, the market price is either settled or disputed.

The problem is that `UMALayerZeroBridge.setOptimisticOracleV3()` allows the owner of the bridge to change the `optimisticOracleV3Address`. This would allow them to use settle or dispute any pending assertion on the market side. Given the permissionless nature of the bridge setup, this issue poses a significant risk for all connected market groups.

RECOMMENDATION

Consider removing the `setOptimisticOracleV3()` function.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Dispute Is Blocked When Contract Is Underfunded

CATEGORY Logical Error
LOCATION `UMALayerZeroBridge.sol`: 165
REVIEW Main Review

DESCRIPTION

`UMALayerZeroBridge` has to pay native token fees to LayerZero when it sends crosschain messages. That's why it inherits from `ETHManagement` which allows users to deposit funds in the contract. It also emits events when the balance of the contract falls below given thresholds. Because of that, we can assume the contract will be regularly funded. However, it's still possible to have periods where there aren't enough funds to pay the LZ message. In these cases all UMA transactions will revert.

This is problematic because `OptimisticOracleV3.disputeAssertion()` calls `UMALayerZeroBridge.assertionDisputedCallback()`. If the bridge is underfunded, the whole dispute transaction will revert. By the time the contract's funded again, the dispute window may have already passed, which results in invalid price being used by the market.

RECOMMENDATION

Consider implementing a retry mechanism where if the balance of the contract is not enough, the transaction doesn't revert, but instead keeps the message and allows it to be sent again in the future. By implementing this approach, disputes will not be blocked and once the bridge gets funded, they will be executed on the market side as well.

RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Binary Markets Cannot Resolve Early

CATEGORY Warning
LOCATION UMASettlementModule.sol: 133
REVIEW Main Review

— DESCRIPTION

In binary markets, outcomes are often known well before the `endTime`. For example, in a market like "Will BTC reach \$100,000 by end of 2024?", the answer becomes clear the moment BTC hits that price. However, current implementation only allows settlement at `endTime`, even if the result is already known. This delays resolution and locks up capital unnecessarily. In contrast, mature platforms like Polymarket allow immediate resolution once the outcome is confirmed.

— RECOMMENDATION

Consider allowing settlement of binary markets before `endTime`.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Ambiguous Token Data

CATEGORY Configuration

LOCATION Market.sol: 131-140

REVIEW Main Review

DESCRIPTION

After `Market.createValid` deploys the virtual tokens, it sorts them in a way that `baseToken` is always the one with the smaller numerical address. However, the two tokens have already been deployed with their names and symbols which hold information if they are base or quote. Because of that it's possible to have base token which is named `Quote Token` and has a symbol `vQuote` and the opposite as well.

RECOMMENDATION

You can expose a new `onlyOwner` function in the `VirtualToken` which changes the name and symbol of the token and use it in `Market` if needed.

RESOLUTION

Status: Acknowledged.

<https://github.com/sapiencexyz/sapience/pull/618>

LOW **L-10**

ACKNOWLEDGED

Market Group Initialized With Arbitrary Nonce

CATEGORY	Events
LOCATION	MarketGroupFactory.sol: 25
REVIEW	Main Review

DESCRIPTION

`MarketGroupFactory.cloneAndInitializeMarketGroup()` will initialize a new market instance and emit the `MarketGroupInitialized` event which accepts a `nonce` argument. This `nonce` is an arbitrary value passed by the user which can lead to wrong event emissions when groups are initialized, unordered nonces, a nonce being used multiple times, etc...

RECOMMENDATION

Implement a per-user nonce tracking mechanism inside the contract.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Potential UMA Price Manipulation By Whales

CATEGORY Validation
LOCATION Market.sol
REVIEW Main Review

— DESCRIPTION

Currently, the `bondAmount` should be greater than the `minBond`, however there is no restriction about how big it can be. To dispute an assertion, a disputer has to first pay the same `bondAmount` that was used at the time of the assertion creation. This allows whales to submit a price assertion with unreasonably big bond that regular users cannot dispute. This is especially dangerous for markets with shorter `assertionLiveness`.

— RECOMMENDATION

Consider implementing a maximum cap for `bondAmount`. This cap should not be lower than the "TVL" of the actual market being settled.

— RESOLUTION

Status: Acknowledged.

LOW **L-12**

RESOLVED

Gas Thresholds Are Checked After Sending The LZ Message

CATEGORY Events

LOCATION MarketLayerZeroBridge.sol, UMLayerZeroBridge.sol

REVIEW Main Review

DESCRIPTION

In both `MarketLayerZeroBridge` and `UMLayerZeroBridge` the call to `_checkGasThresholds()` in `_sendLayerZeroMessageWithQuote()` is executed before the LayerZero message is sent. The contract's balance is higher before sending the message because it still hasn't paid for the fees. This can lead to crossing the threshold, but not emitting an event.

RECOMMENDATION

Consider checking the thresholds after sending the messages.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

LOW **L-13**

RESOLVED

UMA Callbacks Fail After Manual Settlement

CATEGORY Unexpected Behavior
LOCATION UMASettlementModule.sol
REVIEW Main Review

DESCRIPTION

When a market is manually settled via `__manual_setSettlementPrice()`, subsequent UMA settlement callbacks (`assertionResolvedCallback()` and `assertionDisputedCallback()`) will fail due to the `validateUMACallback()` function checking `require(!market.settled, "Market already settled")`. This prevents the bond from being returned to the asserter or disputer, effectively locking their funds.

RECOMMENDATION

Consider either warning the users of this possibility to lose their bond or adding early returns to the two callbacks if the market is settled.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/631>

LOW **L-14**

RESOLVED

Duplicate Event Emission During Balance Update

CATEGORY	Events
LOCATION	MarketLayerZeroBridge.sol: 76-L84
REVIEW	Main Review

DESCRIPTION

The `MarketLayerZeroBridge` contract emits the same `BondWithdrawn` event for two different operations: `CMD_FROM_ESCROW_INTENT_TO_WITHDRAW` and `CMD_FROM_ESCROW_WITHDRAW`. This creates confusion for event listeners and makes it difficult to distinguish between withdrawal intent creation and actual withdrawal execution.

RECOMMENDATION

Create a new event for intents and emit it instead of `BondWithdrawn`.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

LOW **L-15**

ACKNOWLEDGED

Timestamp is not enforced in UMA claim

CATEGORY Best Practices

LOCATION UMASettlementModule.sol: 139-151

REVIEW Main Review

DESCRIPTION

`UMASettlementModule.getClaim()` constructs the asserted claim by using `claimStatementYesOrNumeric` and `claimStatementNo`, but it doesn't append the start or end date of the market. According to step 5 from the `Implementation` section in [UMIP-170] (<https://github.com/UMAprotocol/UMIPs/blob/master/UMIPs/umip-170.md#implementation>), if the assertion was disputed and the timestamp is not found in the claim itself, the voters will use the `block.timestamp` at the moment when `assertTruth()` was called. This is problematic because the message goes through LayerZero and there will be a certain delay before it's executed. By that time, the result of the vote may already have changed, which leads to wrong dispute result.

RECOMMENDATION

Add the `endTime` of the market to the claim.

RESOLUTION

Status: Acknowledged.

LOW **L-16**

RESOLVED

FeeCollectorNFT Is Transferrable And Can Not Be Burned

CATEGORY	Trust Assumptions
LOCATION	FeeCollectorNFT.sol, LiquidityModule.sol
REVIEW	Main Review

— DESCRIPTION

The current `FeeCollectorNft` contract allows NFTs to be freely transferred and does not provide a way to burn (revoke) them. This poses a security and governance risk: the NFT could be transferred to an untrusted party, or remain active even if the fee collector becomes malicious or needs to be shut off.

In addition, if a fee collector becomes inactive or it simply doesn't provide the needed funds, the owner of the market group won't be able to deposit liquidity on their behalf if they are not a fee collectors themselves.

— RECOMMENDATION

1. Override the `_transfer` function so it's not possible to transfer the NFT
2. Add a public `burn()` function that's guarded by the `onlyOwner` modifier and add make it callable by the owner of each MarketGroup, so they are able to revoke fee collector access in case it's needed.
3. Consider allowing the owner of the market group to call `depositCollateral` as well in case they need to do it on behalf of the fee collector.

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/632>

LOW **L-17**

ACKNOWLEDGED

Changing The NonFungiblePositionManager Address Breaks The Protocol

CATEGORY Configuration
LOCATION MarketGroup.sol: 83-92
REVIEW Main Review

— DESCRIPTION

The `ConfigurationModule.updateMarketGroup()` function allows changing the `marketParams` of the current group, including the `uniswapPositionManager`. However, changing the manager means losing access to all of the current LP positions, since the new one won't have ownership over them.

In addition, the project is mixing `market.marketParams.uniswapPositionManager` and `marketGroup.marketParams.uniswapPositionManager`. For example, `decreaseLiquidityPosition()` calls `Pool.getCurrentPositionTokenAmounts` and `Pool` uses the market `uniswapPositionManager`, while later in the flow the new one will be used.

Places where `market.marketParams.uniswapPositionManager` is used:

- `Pool.getCurrentPositionTokenAmounts()`
- `SettlementModule._settleLiquidityPosition()`

— RECOMMENDATION

Consider adding a restriction to not change the `uniswapPositionManager` when updating the parameters.

— RESOLUTION

Status: Acknowledged.

LOW **L-18**

ACKNOWLEDGED

Trading Is Allowed Before Market Start Time

CATEGORY Unexpected Behavior
LOCATION TradeModule.sol
REVIEW Main Review

DESCRIPTION

Although each market includes both a `startTime` and an `endTime`, the protocol does not enforce those bounds. In its current form, trading and liquidity-provision actions may occur well before the declared `startTime`. Conversely, if someone maliciously sets `endTime = startTime + 1` second, the market becomes eligible for “manual settlement” just two seconds after its supposed launch window, even though positions could have been opened long before. In the opposite scenario, if a market is created with a `startTime` in the past, the manual settlement path via `__manual_setSettlementPrice` may never become available at all. This disconnect allows both out-of-window activity and incorrect settlement timing.

RECOMMENDATION

Enforce that `startTime` must be strictly in the future when a market is created, and add a precondition in every trading and liquidity function requiring `block.timestamp >= startTime`. This guarantees that no positions can be opened or modified before the intended launch, and that settlement timing accurately reflects the market's defined window.

RESOLUTION

Status: Acknowledged.

LOW **L-19**

ACKNOWLEDGED

Insufficient Minimum Collateral Requirement

CATEGORY Validation
LOCATION Position.sol: 109-111
REVIEW Main Review

DESCRIPTION

`Position.afterTradeCheck()` enforces that the minimum deposited collateral for a position is at least `minTradeSize`, but because `minTradeSize` is in `base` terms, there are two potential problems:

- If the price difference between `base` and `quote` is too large, the check may be inefficient
- The comparison is done in 18 decimals, but the collateral token may have less decimals. For example, `minTradeSize = 10000` would guarantee only 1 wei of actual collateral if the collateral is with 6 decimals, because then `depositedCollateralAmount = 1e12 > 10000`.

RECOMMENDATION

Consider using a different approach for setting minimum collateral requirement.

RESOLUTION

Status: Acknowledged.

LOW **L-20**

ACKNOWLEDGED

Extra Collateral Can Be Withdrawn

CATEGORY Warning
LOCATION Trade.sol: 261-266
REVIEW Main Review

— DESCRIPTION

If a position being modified in `Trade.quoteOrTrade` incurs a loss greater than the deposited collateral, the following lines are executed:

```
if (collateralLoss > params.oldPosition.depositedCollateralAmount) {  
  // If the collateral to return is more than the deposited collateral, then the position is in  
  a loss  
  // and the collateral should be reduced to zero  
  output.position.depositedCollateralAmount = 0;  
  extraCollateralRequired = collateralLoss - params.oldPosition.depositedCollateralAmount;  
}
```

This will reset the deposited collateral to 0 and force the user to pay the rest as extra collateral. The problem is that this collateral will be attributed to the position's `depositedCollateral` again. The user can just modify the position again after that and recalculate the `newPositionCollateralRequired`, which will allow them to get back the extra collateral or use it to back their new position.

— RECOMMENDATION

This case should never happen in practice because the protocol limits the price range of the virtual tokens. One way to fix it is to not add the extra collateral towards the `depositedCollateralAmount`.

— RESOLUTION

Status: Acknowledged.

LOW **L-21**

ACKNOWLEDGED

Market Initial Price Can Be Changed

CATEGORY MEV
LOCATION Market.sol: 147
REVIEW Main Review

— DESCRIPTION

When a market is created via `Market.createValid()`, its pool is initialized with `startingSqrtPriceX96`. However, since there is no liquidity added, anyone can move that price to whatever new price they desire, even if its not in the pool's allowed range.

This can be utilized by a first liquidity provider to try sandwiching future LPs with no slippage protections:

- Move the price to an extreme end
- Honest LP adds liquidity at unfavorable price
- Trade against honest LP and profit

— RECOMMENDATION

If the honest LP provides reasonable slippage parameters, they should be safe, so make sure that LPs are aware of this possibility.

— RESOLUTION

Status: Acknowledged.

LOW **L-22**

ACKNOWLEDGED

Claim Max. Length Is Not Restricted

CATEGORY	Validation
LOCATION	Market.sol: 62-173
REVIEW	Main Review

— DESCRIPTION

During market creation via `Market.createValid()`, market owners can specify arbitrarily long claim messages that will be used for truth assertions during settlement. However, the protocol does not enforce any maximum length restriction on these claims at creation time.

This can create an issue during the settlement phase: when the settlement process attempts to send the claim data through LayerZero's cross-chain messaging infrastructure, it may exceed the `maxMessageSize` limit imposed by the LayerZero libraries, which will lead to inability to settle the market.

— RECOMMENDATION

Apply a maximum length check for the claims.

— RESOLUTION

Status: Acknowledged.

LOW **L-23**

ACKNOWLEDGED

assertTruth Can Revert

CATEGORY	DoS
LOCATION	UMALayerZeroBridge.sol
REVIEW	Main Review

— DESCRIPTION

Since `OptimisticOracleV3.assertTruth()` doesn't store any information about `msg.sender` when computing the assertion id, a malicious user can use the same information right before the `UMALayerZeroBridge` calls `assertTruth()`. This will result in the LZ transaction reverting and creating a risk of blocking the settlement of the market. For example, if the transaction is not retried and a withdrawal request is executed, the next time the assertion is retried, a revert can happen because of underflow in `_updateBondBalance()`.

This action is not free though, the malicious user has to pay the bond with no possibility of getting it back, which makes the attack highly unlikely, but still something to keep in mind.

— RECOMMENDATION

Document the risk and check regularly for failed messages.

— RESOLUTION

Status: Acknowledged.

LOW **L-24**

RESOLVED

0 Value Transfers Are Allowed in `updateCollateral()`

CATEGORY Unexpected Behavior
LOCATION `Position.sol`: 96
REVIEW Main Review

DESCRIPTION

`Position.updateCollateral()` checks if the `deltaCollateral` is 0 and skips any transfer logic if it's. However, this value is in 18 decimals. If it's negative, `denormalizeCollateralAmount()` will be applied to it, which means it's possible for the end result to end up being 0. Then, `collateralAsset.safeTransfer(msg.sender, 0);` will be triggered. If the collateral token reverts on 0 transfers, all actions using `updateCollateral` are in a risk of DOS.

RECOMMENDATION

Skip the token transfer if the end amount is 0.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/632>

LOW L-25

ACKNOWLEDGED

assertionResolvedCallback Function Ignores assertedTruthfully

CATEGORY Logical Error
LOCATION UMASettlementModule.sol
REVIEW Main Review

DESCRIPTION

`UMASettlementModule.assertionResolvedCallback(bytes32 assertionId, bool)` is the final hook that receives the `OptimisticOracleV3` (OOv3) verdict.

The function currently performs the following logic (simplified):

```
function assertionResolvedCallback(bytes32 assertionId, bool) external {  
    MarketGroup.Data storage marketGroup = MarketGroup.load();  
    uint256 marketId = marketGroup.marketIdByAssertionId[assertionId];  
    Market.Data storage market = Market.load(marketId);  
    validateUMACallback(market, msg.sender, assertionId);  
    Market.Settlement storage settlement = market.settlement;  
    if (!market.settlement.disputed) {  
        market.setSettlementPriceInRange(DecimalPrice.sqrtRatioX96ToPrice(settlement.settlementPriceSqrtX96));  
        emit MarketSettled(marketId, assertionId, settlement.settlementPriceSqrtX96);  
    }  
    // clear the assertionId  
    market.assertionId = bytes32(0);  
}
```

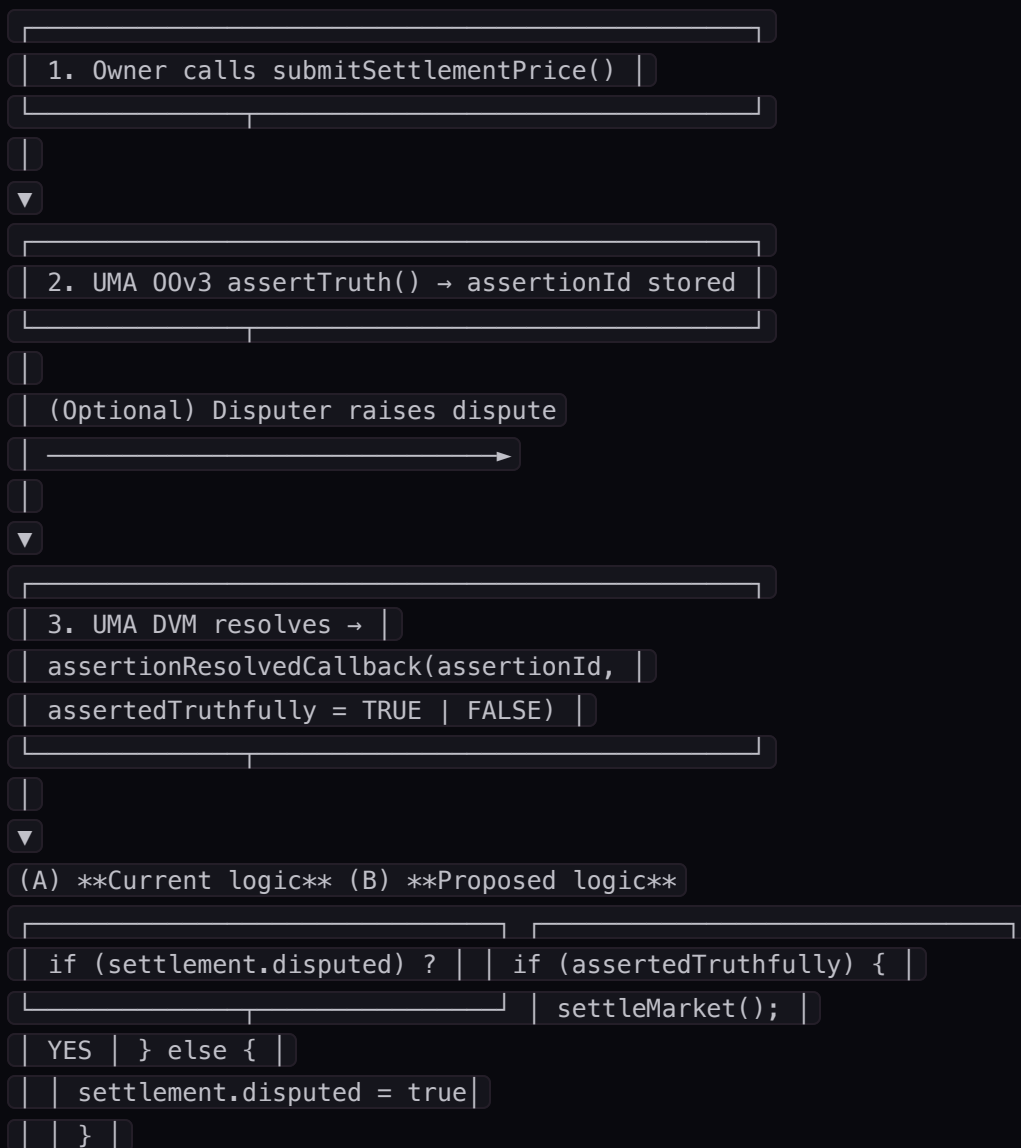
- The boolean `assertedTruthfully` is completely ignored.
- The only gate for calling `setSettlementPriceInRange` is when `market.settlement.disputed` is `false`.

L-25 assertionResolvedCallback Function Ignores assertedTruthfully

CONTINUED

Once a single dispute has been raised, no matter the final oracle ruling, `disputed` is locked to `true`, so the market never settles (`market.settled` remains `false`). This behavior is fundamentally incorrect: once the DVM has accurately resolved an assertion, the protocol should accept that outcome and finalize the settlement without requiring further manual intervention. Ignoring the `assertedTruthfully == true` signal undermines the integrity and purpose of the optimistic oracle, turning a one-time verification into an endless manual loop and removing trust in the oracle's result.

A malicious actor can repeatedly dispute every settlement submission, freezing the market indefinitely even if the UMA DVM ultimately rules the claim true, forcing the owner to perform a manual settlement and incur gas and operational overhead, while the attacker's cost remains capped at a single bond per round.



L-25 assertionResolvedCallback Function Ignores assertedTruthfully

CONTINUED

```

▼ [redacted]
[redacted]
| 4A. *Skip* settleMarket() | |
| (disputed == true) | |
| - market.settled = false | |
| - assertionId cleared | |
[redacted]
| |
| ← [redacted] |
| | |
| Market forever | |
| UNSETTLED ← [redacted] |
| |
▼ ▼
(owner must fall back to [redacted]
manual settlement path) | 4B. settleMarket() |
| - market.settled |
| = true |
| - collateral is |
| unlocked |
[redacted]

```

— RECOMMENDATION

Consider the `assertedTruthfully` flag and settle the market whenever the oracle rules in favour of the `asserter`, even if a dispute occurred:

```

function assertionResolvedCallback(bytes32 assertionId, bool assertedTruthfully) external {
    MarketGroup.Data storage mg = MarketGroup.load();
    uint256 marketId = mg.marketIdByAssertionId[assertionId];
    Market.Data storage market = Market.load(marketId);
    validateUMACallback(market, msg.sender, assertionId);
    Market.Settlement storage settlement = market.settlement;
    if (assertedTruthfully) {
        // Oracle confirmed the submitted price – finalise settlement
        market.setSettlementPriceInRange(
            DecimalPrice.sqrtRatioX96ToPrice(settlement.settlementPriceSqrtX96)
        );
        emit MarketSettled(marketId, assertionId, settlement.settlementPriceSqrtX96);
    }
}

```

L-25 assertionResolvedCallback Function Ignores assertedTruthfully

CONTINUED

```
} else {  
  // Oracle rejected the price – leave the market unsettled  
  settlement.disputed = true;  
}  
// Clear assertion so a new one can be posted if needed  
market.assertionId = bytes32(0);  
}
```

— RESOLUTION

Status: Acknowledged.

<https://github.com/sapiencexyz/sapience/pull/631>

INFORMATIONAL I-01

RESOLVED

Use Call Instead Of Transfer For Native Assets Transfers

CATEGORY Warning
LOCATION ETHManagement.sol
REVIEW Main Review

DESCRIPTION

The `transfer` function on `payable` addresses forwards a fixed 2300 gas stipend to the recipient and reverts on failure, which can lead to unexpected reverts if the recipient is a contract consuming more gas in its fallback/receive function (e.g., due to logging or checks). Some multi-signature wallets require more than 2300 gas to receive a native asset transfer. For example, Gnosis Safe supports forwarding via fallback. Using `transfer()` with a Safe's multi-sig can lead to gas depletion and failed transfers.

In this codebase, the a raw `transfer` call is used in the abstract `ETHManagement.withdrawETH` function (inherited by bridges like `UMALayerZeroBridge` and `MarketLayerZeroBridge`), where ETH is sent to the owner:

```
payable(owner()).transfer(amount);
```

If the owner becomes a contract (e.g., Gnosis Safe's multi-sig) withdrawals could revert.

RECOMMENDATION

Replace `transfer` with a low-level `call{value: amount}("")`, check success via a bool, and handle failures gracefully (e.g., revert with custom error).

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL I-02

RESOLVED

Debug Statements In Production Code

CATEGORY Best Practices
LOCATION UMLayerZeroBridge.sol
REVIEW Main Review

DESCRIPTION

The `UMLayerZeroBridge.sol` contract includes an import statement for `console2` from `forge-std/console2.sol`, as shown below:

```
import {console2} from "forge-std/console2.sol";
```

This import is a debugging utility typically used in development environments (e.g., Foundry testing) for logging purposes, but it is not intended for production code.

RECOMMENDATION

Remove the unused `import {console2} from "forge-std/console2.sol";` statement from `UMLayerZeroBridge.sol`.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL I-03

RESOLVED

Unused Parameters

CATEGORY	Best Practices
LOCATION	Market.sol, BridgeTypes.sol
REVIEW	Main Review

DESCRIPTION

The `Market.Data` struct in `Market.sol` declares a mapping `mapping(uint256 => Debt.Data) lpDebtPositions` intended to track liquidity provider debt positions, but it is never read from or written to anywhere in the codebase. This is evident from a full code search showing no accesses to this field beyond its declaration. The struct is defined as:

```
struct Data {
    uint256 startTime;
    uint256 endTime;
    int24 baseAssetMinPriceTick;
    int24 baseAssetMaxPriceTick;
    VirtualToken baseToken;
    VirtualToken quoteToken;
    IUniswapV3Pool pool;
    bool settled;
    uint256 settlementPriceD18;
    mapping(uint256 => Debt.Data) lpDebtPositions; //<-----
    bytes32 assertionId;
    Settlement settlement;
    ISapienceStructs.MarketParams marketParams;
    uint160 sqrtPriceMinX96;
    uint160 sqrtPriceMaxX96;
    uint256 minPriceD18;
    uint256 maxPriceD18;
    uint256 feeRateD18;
    uint256 id;
    bytes claimStatementYesOrNumeric;
    bytes claimStatementNo;
};
```

On the other hand, something similar occurs in the `SettlementData` struct with the `uint256 epochId` variable:

I-03 Unused Parameters

CONTINUED

```
struct SettlementData {  
    address market;  
    uint256 epochId; //<-----  
    uint256 settlementPrice;  
    uint256 timestamp;  
}
```

Unused storage like this wastes deployment gas and increases contract size without functionality, complicating future maintenance or upgrades.

— RECOMMENDATION

Remove the unused `lpDebtPositions` mapping from the `Market.Data` struct and the `epochId` parameter from the `SettlementData` struct to reduce contract size and eliminate dead code.

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL I-04

PARTIALLY RESOLVED

Lack Of A Double Step Transfer Ownership Pattern

CATEGORY	Best Practices
LOCATION	VirtualToken.sol, MarketGroup.sol, FeeManagement.sol
REVIEW	Main Review

DESCRIPTION

The current ownership transfer process for all the contracts inheriting from the `Ownable` contract involves the current owner calling the `transferOwnership` function:

```
function transferOwnership(address newOwner) public virtual onlyOwner {  
    require(newOwner != address(0), "Ownable: new owner is the zero address");  
    _transferOwnership(newOwner);  
}
```

If the nominated EOA account is not a valid account, it is entirely possible that the owner may accidentally transfer ownership to an uncontrolled account, losing the access to all functions with the `onlyOwner` modifier.

RECOMMENDATION

It is recommended to implement a two-step process transfer ownership process where the owner nominates an account and the nominated account needs to call an `acceptOwnership` function for the transfer of the ownership to fully succeed. This ensures the nominated EOA account is a valid and active account. This can be easily achieved by using [OpenZeppelin's Ownable2Step] (<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/access/Ownable2Step.sol>) contract.

RESOLUTION

Status: Partially resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL I-05

RESOLVED

Missing Events for Configuration Setter Functions

CATEGORY	Events
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

In DeFi protocols, setter functions that modify critical configuration parameters should emit events to enable off-chain monitoring, and alerting for changes (e.g., by indexers like The Graph or frontend apps). This ensures transparency and helps detect unauthorized or erroneous updates. However, several setter functions across the codebase do not emit events, making it difficult to track modifications without scanning transaction logs manually or relying on indirect effects. The affected functions include:

- `UMALayerZeroBridge.setOptimisticOracleV3(address _optimisticOracleV3)` : Updates the UMA oracle address, a core dependency for assertions, but no event is emitted to log the new address.
- `UMALayerZeroBridge.setLzReceiveCost(uint128 _lzReceiveCost)` : Sets LayerZero receive gas cost, impacting fee calculations, without emission.
- `UMALayerZeroBridge.setGasThresholds(uint256 _warningGasThreshold, uint256 _criticalGasThreshold)` : Configures gas alerts, no event.
- `MarketLayerZeroBridge.setLzReceiveCost(uint128 _lzReceiveCost)` : Similar to above, no event.
- `MarketLayerZeroBridge.setGasThresholds(uint256 _warningGasThreshold, uint256 _criticalGasThreshold)` : No event.
- `MarketLayerZeroBridge.enableMarketGroup(address marketGroup)` and `disableMarketGroup(address marketGroup)` : Toggles market group access, critical for operations, but no events to track enables/disables.

Other setters (e.g., `setBridgeConfig` in both bridges) do emit, showing inconsistency.

RECOMMENDATION

Add dedicated events (e.g., `OptimisticOracleV3Updated(address newOracle)`) to all listed setters, emitting old/new values where applicable for auditability.

I-05 Missing Events for Configuration Setter FunctionsCONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL I-06

RESOLVED

Inconsistent Storage Slot Naming in ERC721 Implementation

CATEGORY	Warning
LOCATION	ERC721EnumerableStorage.sol, ERC721Storage.sol
REVIEW	Main Review

DESCRIPTION

The Sapience protocol's ERC721 storage slots in `ERC721Storage` and `ERC721EnumerableStorage` libraries, are hashed using a namespace prefix tied to `"io.synthetix.core-contracts"` which appears to be a remnant from forked or imported Synthetix code. Specifically, the constants `_SLOT_ERC721_STORAGE` and `_SLOT_ERC721_ENUMERABLE_STORAGE` employ keccak256 hashes of strings like `"io.synthetix.core-contracts.ERC721"` and `"io.synthetix.core-contracts.ERC721Enumerable"` respectively, to derive their slots. This is mismatched with the Sapience protocol's branding.

RECOMMENDATION

Consider updating the storage slot constants to use a Sapience-specific namespace, such as hashing `"io.sapience.core-contracts.ERC721"` for `_SLOT_ERC721_STORAGE` and `"io.sapience.core-contracts.ERC721Enumerable"` for `_SLOT_ERC721_ENUMERABLE_STORAGE`.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL I-07

ACKNOWLEDGED

Protocol Do Not Support Fee On Transfer Tokens

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Sapience, which relies on precise collateral transfers for maintaining position solvency in its trading and liquidity modules, does not accommodate ERC20 tokens with built-in transfer fees. This limitation happens primarily in functions handling collateral deposits, such as `createTraderPosition` and `modifyTraderPosition` in `TradeModule`, as well as the underlying `updateCollateral` in `Position`, where amounts are normalized/denormalized based on the token's decimals and transferred via OpenZeppelin's `safeTransferFrom`. For standard tokens, this ensures the protocol receives the full intended amount, but with fee-on-transfer tokens, the recipient (the protocol contract) gets less than requested, e.g., a 2% fee on a 100-unit transfer yields only 98 units, while the sender's balance deducts the full 100. This discrepancy can lead to undercollateralization: the position's `depositedCollateralAmount` is updated assuming the full normalized value arrived, but the actual balance is short, allowing insolvent positions to pass validation checks in `afterTradeCheck` or `validateCollateralRequirementsForTrade`. Essentially, the protocol is not prepared to support fee-on-transfer tokens as collateral.

RECOMMENDATION

Document the non-support for fee-on-transfer tokens in the protocol documentation to inform users and prevent incompatible asset usage.

RESOLUTION

Status: Acknowledged.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL **I-08**

RESOLVED

Reference To Deprecated vGas Tokens In Error Message

CATEGORY	Warning
LOCATION	TradeModule.sol, LiquidityModule.sol
REVIEW	Main Review

DESCRIPTION

In the `TradeModule` contract, the closure logic within `modifyTraderPosition` includes validation checks to ensure no residual vBase tokens remain before finalizing a position close, which is crucial for maintaining single-sided token states and preventing invalid settlements. However, the revert error messages incorrectly reference "vGas tokens" in both cases, if `position.vBaseAmount > 0` or `position.borrowedVBase > 0`, despite the code and protocol using "vBase" (the virtual base token representing the variable outcome asset). This discrepancy stems from legacy naming conventions in an earlier protocol version where "vGas" might have been a placeholder or prior term for vBase.

The `LiquidityModule` does also contain some references to this old code.

RECOMMENDATION

Update the error strings in the revert statements to accurately reference "vBase tokens" (e.g., "Cannot close position with vBase tokens" and "Cannot close position with borrowed vBase tokens") to align with the current protocol terminology.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL I-09

ACKNOWLEDGED

Omission of Explicit Security and Executor LZ Configurations

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The Sapience protocol's LayerZero bridge contracts (`UMALayerZeroBridge` and `MarketLayerZeroBridge`), built on the OApp framework for cross-chain interactions, do not include explicit setups for security parameters (e.g., Destination Verification Networks or DVNs) or executor configurations. As outlined in LayerZero V2 documentation, OApps without custom configurations automatically revert to default settings provided by LayerZero Labs, encompassing a predefined suite of DVNs for message authentication and standard executor behaviors for handling receipts. These defaults are engineered for broad compatibility and security in typical deployments, leveraging LayerZero's rigorously audited verifiers and operational baselines to ensure reliable message delivery and verification. However, they may not be optimally tailored to Sapience's unique requirements, such as elevated verification thresholds for sensitive assertion callbacks in prediction markets.

— RECOMMENDATION

Merely informative. While the current approach can be considered secure, do consider defining the OApp's security parameters (e.g., DVN assignments) and executor configurations during the bridge setup to customize beyond defaults, in order to ensure optimal verification and execution for Sapience's cross-chain flows and document any reliance on defaults for clarity in protocol maintenance.

— RESOLUTION

Status: Acknowledged.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL **I-10**

ACKNOWLEDGED

Absence of Excess Collateral Refund in LP Position Creation

CATEGORY	Logical Error
LOCATION	LiquidityModule.sol
REVIEW	Main Review

DESCRIPTION

In the `createLiquidityPosition` function within the `LiquidityModule` contract, users provide a `collateralAmount` that is normalized and fully locked into the position's `depositedCollateralAmount` via `position.updateCollateral(totalDepositedCollateralAmount)`, even if it exceeds the computed `requiredCollateralAmount` from `market.requiredCollateralForLiquidity`. This differs from `decreaseLiquidityPosition`, where excess is refunded if `newCollateralAmount > requiredCollateralAmount` through a conditional `updateCollateral(required)`. As a result, any over-deposit (common due to uncertainty in exact requirements influenced by pool state and range) remains locked until a future decrease or close, inefficiently tying up capital. For example, if a user deposits 1000 units but only 800 are required, the full 1000 is held, preventing redeployment elsewhere. The logic in `updateValidLp` computes but doesn't act on excess (beyond revert for shortfall, bypassed for fee collectors).

RECOMMENDATION

Implement a refund mechanism in `createLiquidityPosition` mirroring `decreaseLiquidityPosition` by adding a conditional release after `updateValidLp`. Modify the collateral handling as follows:

```
(requiredCollateralAmount, totalDepositedCollateralAmount,,) = position.updateValidLp(
    market,
    Position.UpdateLpParams({
        uniswapNftId: uniswapNftId,
        liquidity: liquidity,
        additionalCollateral: normalizedCollateral,
        additionalLoanAmount0: addedAmount0,
        additionalLoanAmount1: addedAmount1,
        lowerTick: params.lowerTick,
        upperTick: params.upperTick,
```

I-10 Absence of Excess Collateral Refund in LP Position Creation

CONTINUED

```
tokensOwed0: 0,  
tokensOwed1: 0,  
isFeeCollector: marketGroup.isFeeCollector(msg.sender)  
})  
);  
int256 deltaCollateral;  
if (totalDepositedCollateralAmount > requiredCollateralAmount) {  
    deltaCollateral = position.updateCollateral(requiredCollateralAmount);  
    totalDepositedCollateralAmount = requiredCollateralAmount; // Adjust for event  
} else {  
    deltaCollateral = position.updateCollateral(totalDepositedCollateralAmount);  
}
```

This refunds excess via safe transfer in `updateCollateral`, updates the locked amount and ensures consistency.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

RESOLVED

Unused Code

CATEGORY	Superfluous Code
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The following code is not used or missing implementation

- Library `Errors`: `OnlyInitializer` and `InvalidTickSpacing` errors are not emitted.
- Library `DecimalMath`: Imported in `TradeModule` and `ViewsModule` but never used.
- Struct `SettlementData`: declared in `BridgeTypes` but not used.
- `Ownable`; contract inherited from `VirtualToken` but `onlyOwner` is not used.

RECOMMENDATION

Remove the unused code and imports.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/632>

INFORMATIONAL I-12

RESOLVED

Missing Access Control For Virtual Token

CATEGORY Access Control
LOCATION VirtualToken.sol: 11
REVIEW Main Review

DESCRIPTION

The `VirtualToken.mint` function has no access control. Although `type(uint256).max` is minted during market deployment preventing new minted tokens, users can still call it with zero amount and emit a `Transfer` from the zero address.

RECOMMENDATION

Consider adding `onlyOwner` modifier to the `mint` function, or remove the Ownable inherited contract.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL **I-13**

RESOLVED

Unnecessary Returned Values

CATEGORY	Superfluous Code
LOCATION	UMALayerZeroBridge.sol
REVIEW	Main Review

DESCRIPTION

The UMALayerZeroBridge `assertionResolvedCallback` and `assertionDisputedCallback` functions can only be called by OOV3, which does not expect any return values.

RECOMMENDATION

The return value `MessagingReceipt memory` should be avoided as it only wastes execution gas.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL **I-14**

RESOLVED

Deadline Is Validated Twice

CATEGORY Superfluous Code
LOCATION LiquidityModule.sol
REVIEW Main Review

DESCRIPTION

Uniswap `NonfungiblePositionManager` liquidity actions require a `deadline` param, provided by the user. Transaction will revert in case `block.timestamp > deadline`.

However, the `LiquidityModule` also validates this deadline, spending unnecessary gas.

RECOMMENDATION

Remove the `deadline` check as this is handled by the `NonfungiblePositionManager`.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/632>

INFORMATIONAL I-15

RESOLVED

Overwritten Returned Parameters

CATEGORY Logical Error
LOCATION LiquidityModule.sol: 273
REVIEW Main Review

DESCRIPTION

During `closeLiquidityPosition`, the `decreasedAmount0` and `decreasedAmount1` are assigned to the values returned by the `NonfungiblePositionManager.decreaseLiquidity` call.

However, these parameters are overwritten by the returned values of the internal `_closeLiquidityPosition`, which include all collected amounts, not just the decrease liquidity amounts.

RECOMMENDATION

If this is expected behavior, consider not storing the returned values of `NonfungiblePositionManager.decreaseLiquidity` as they are not used and is only wasting gas.

Alternatively, if the idea was to only return liquidity amounts decreased, do not overwrite them with the `return` statement.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/632>

INFORMATIONAL I-16

RESOLVED

Unused Param In _sendBalanceUpdate

CATEGORY	Gas Optimization
LOCATION	BondManagement.sol: 50
REVIEW	Main Review

DESCRIPTION

The `finalAmount` param in `_sendBalanceUpdate` is never used and could be removed for optimization. Additionally, in `depositBond`, this `finalAmount` is set as the pre-deposit bond balance, which may be confusing to the user or integrators.

RECOMMENDATION

Remove the `finalAmount` parameter completely.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL **I-17**

RESOLVED

Missing INftModule In ISapience

CATEGORY Informational
LOCATION ISapience.sol
REVIEW Main Review

DESCRIPTION

The `ISapience` interface contains all module's functionality except for the `NftModule`, in order to execute ERC721 related functions.

RECOMMENDATION

Create the `INftModule` and inherit from it in `ISapience`.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL I-18

RESOLVED

Inefficient Usage of DURATION_MULTIPLIER

CATEGORY	Gas Optimization
LOCATION	SettlementModule.sol: 74
REVIEW	Main Review

DESCRIPTION

In `__manual_setSettlementPrice`, the `DURATION_MULTIPLIER` is hardcoded as `2`. However, it is not stored as a constant or variable, which is suboptimal from both a gas efficiency and configurability standpoint.

If this multiplier is intended to be fixed, consider declaring it as a `constant` or `immutable` variable to benefit from gas savings. If it is meant to be adjustable, store it as a regular `uint256` state variable and implement a setter function restricted to the contract owner or admin.

RECOMMENDATION

- If fixed: Declare as `uint256 constant DURATION_MULTIPLIER = 2;`
- If configurable: Store it as a state variable with appropriate setter access control.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/631>

INFORMATIONAL I-19

RESOLVED

Layer Zero Refunds Trigger ETHDeposited Event

CATEGORY	Events
LOCATION	ETHManagement.sol: 58
REVIEW	Main Review

DESCRIPTION

When sending LZ messages, the bridge contract is used as the refund recipient. Although there the bridge fee is calculated before sending the cross-chain message, any refund will execute the `receive` function, emitting the `ETHDeposited`. This event is only expected when contract is funded with ETH by depositors, which may create confusion for offchain services.

RECOMMENDATION

Document this behavior to make sure offchain services are aware of LZ refunds, as they are not actual deposits.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL I-20

ACKNOWLEDGED

Avoid Using `_lzSend`

CATEGORY Gas Optimization
LOCATION MarketLayerZeroBridge.sol: 138
REVIEW Main Review

DESCRIPTION

The `_lzSend` function relies on the `msg.value` to pay the bridge fee. Therefore, the bridge contract have to invoke `this._sendMessageWithETH` with an ether value in order to make it work.

Although is a good practice to use the available `_lzSend`, this is just a helper function to verify `msg.value > nativeFee` before calling `endpoint.send`. As the contract does not receive ether from users when sending a message (i.e. during `depositBond`), the external call can be avoided to save gas.

RECOMMENDATION

Consider manually validating validating `msg.value > nativeFee` and calling `endpoint.send` directly to avoid the external call.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-21

ACKNOWLEDGED

Quote Functions Use Normalized Collateral

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The `quote` functions are helper functions to determine the required collateral for trades or liquidity positions.

However, the collateral amount handling is inconsistent when the scaling factor is greater than 1. For example:

- In `quoteLiquidityPositionTokens`, the `depositedCollateralAmount` needs to be normalized in order to return correct values
- In `quoteCreateTraderPosition` the `requiredCollateral` needs to be denormalized in order to know the actual collateral in the original token values.

This not only creates bad UX but can introduce rounding issues if the actual amount needs to be divided by the scaling factor.

RECOMMENDATION

Quote function should accept and return the denormalized collateral amounts, so traders and liquidity providers can use quote functions without modifications to the amounts.

RESOLUTION

Status: Acknowledged.

<https://github.com/sapiencexyz/sapience/pull/632>

INFORMATIONAL I-22

RESOLVED

Closing Liquidity DoS'ed By Dust Amounts

CATEGORY	Rounding
LOCATION	LiquidityModule.sol: 559
REVIEW	Main Review

DESCRIPTION

The `feeCollector` feature allows these users to create liquidity position without depositing collateral.

Due to rounding on the Uniswap side, decreasing liquidity returns 1 wei lower than the amount originally added. During position close flow, we decrease all liquidity and collect tokens from Uniswap and add 1 wei to collected amounts to fix the rounding issue.

However, in case the fee collector's liquidity position was entirely in `vQuote` (added liquidity to lower range), the following scenario can arise:

- add single sided liquidity in `vQuote`, `borrowedVQuote` = X
- decrease liquidity once
- close position, amount collected is X - 2 wei.
- Add 1 wei to `collectedAmount1` to fix rounding issue
- Final position amounts: `vQuoteAmount` = X, `borrowedVQuote` = X - 1

Due to the fact that `borrowedVQuote > 0` while `vBase` amounts are 0, position is not entirely closed, and converted into a `Trade` position. When attempting to close the trade, it reverts with `InvalidData("At least one token should be traded")` as the position size is `vBaseAmount - vBorrowedVBase = 0`

RECOMMENDATION

Consider removing the `feeCollector` feature, as not only this leads to rounding errors like the one described above, but also invalidates certain solvency assertions in the protocol if these users are allowed to create positions without collateral.

Alternatively, only attempt to close the trade position if position size (net `vBase`) is non zero.

RESOLUTION

Status: Resolved.

I-22 Closing Liquidity DoS'ed By Dust Amounts

CONTINUED

<https://github.com/sapiencexyz/sapience/pull/632>

INFORMATIONAL I-23

RESOLVED

Incorrect FeeManagement Comment

CATEGORY Informational
LOCATION FeeManagement.sol: 13
REVIEW Main Review

DESCRIPTION

One of the listed actions of the `FeeManagement` contract is:

- `Checking gas thresholds and revert if necessary`

However, the contract only emits events if the thresholds are crossed, no reverts happen.

RECOMMENDATION

Remove the `~~contract~~` comment if the intention is to only emit events.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL **I-24**

ACKNOWLEDGED

Disputes Force Settlement Process To Be Reset

CATEGORY Informational
LOCATION UMASettlementModule.sol
REVIEW Main Review

— DESCRIPTION

As already mentioned in previous rounds of Foil, any user that's willing to pay the price for a dispute can cause valid price settlements to be discarded, causing a restart of the settlement process.

— RECOMMENDATION

Keep this limitation in mind.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-25

ACKNOWLEDGED

getPnl Function May Return Stale Data

CATEGORY Warning
LOCATION Position.sol: 273
REVIEW Main Review

DESCRIPTION

`Position.getPnl()` is a view function which returns the current `PnL` for a given position. If the position is of type `Liquidity`, the `PnL` calculation depends on the `tokens0wed0` and `tokens0wed1` variables which may not have been updated since the last fee accumulation. In result, the `PnL` reported will be lower than the actual one. This will affect `ViewsModule.getPositionPnl()` and `ViewsModule.getPositionCollateralValue()`.

RECOMMENDATION

Mention this lagging behavior explicitly so external integrators are aware of it.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-26

RESOLVED

Manual Settlement Can Not Set The Price To 0

CATEGORY Warning
LOCATION SettlementModule.sol
REVIEW Main Review

DESCRIPTION

`SettlementModule.__manual_setSettlementPrice()` can be executed if the market hasn't been settled for a given time. It sets the price to whatever the current Uniswap price is, but it can't set the price to 0, for example if the market is of type `YES/NO`.

RECOMMENDATION

Keep that in mind.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/631>

INFORMATIONAL **I-27**

RESOLVED

Unused Encoder Function

CATEGORY	Informational
LOCATION	cmdEncoder.sol: 13-15
REVIEW	Main Review

DESCRIPTION

When sending `LayerZero` messages, `Encoder.encodeType()` is not used, the message is constructed using `abi.encode()` instead. The existence of `Encoder.encodeType()` is dangerous because it uses `abi.encodePacked()` which can lead to malformed data if the function is used in the future.

RECOMMENDATION

Either remove the function from the `Encoder` library or change it to `abi.encode()` and use it when sending crosschain messages.

RESOLUTION

Status: Resolved.

<https://github.com/sapiencexyz/sapience/pull/618>

INFORMATIONAL I-28

ACKNOWLEDGED

Note About withdrawCollateral Return Value

CATEGORY	Informational
LOCATION	MarketGroup.sol: 114-128
REVIEW	Main Review

DESCRIPTION

`MarketGroup.withdrawCollateral()` accepts an 18 decimals amount to be withdrawn and returns the withdrawn amount, again in 18 decimals. Because it performs `denormalize()` and then `normalize()` again, if the collateral token has less than 18 decimals, the end result of `withdrawCollateral` will be a smaller value than the initially passed amount, even if there are enough funds in the contract to pay the user.

RECOMMENDATION

Be aware of this behavior.

RESOLUTION

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

Withdrawal Race Condition Causes Balance Desync

CATEGORY	Logical Error
LOCATION	BondManagement.sol: 144
REVIEW	Remediation Review

DESCRIPTION

The fix for H-01 was incomplete. As withdrawal intent on UMA chain is cleared immediately after `executeWithdrawal`, this allows a new `intentToWithdrawBond` to be submitted in the same transaction, leading to a possible race condition and accounting mismatch.

Example scenario:

- Alice holds 1000 tokens on both UMA and Market chains
- She withdraws 600 → intent set to 600 on both chains
- After delay, calls `executeWithdrawal` on UMA chain and immediately submits a new withdrawal intent for 400 in the same transaction
- If `intentToWithdrawBond(400)` is processed on Market chain before `executeWithdrawal(600)`, the withdrawal intent is overwritten with 400
- `executeWithdrawal(600)` then underflows when subtracting from intent, reverting on Market chain
- Alice still receives 600 tokens on UMA chain while retaining 1000 on Market chain

This attack path can also be achieved with the newly implemented `removeWithdrawalIntent`.

RECOMMENDATION

Do not clear withdrawal intent on UMA chain immediately after `executeWithdrawal` and `removeWithdrawalIntent`. Instead, clear it only after `CMD_FROM_ESCROW_WITHDRAW` and `CMD_FROM_ESCROW_REMOVE_WITHDRAWAL_INTENT` is executed on Market chain and a confirmation message is received back on UMA chain. This prevents immediate re-submission of withdrawal intent and eliminates the race condition.

RESOLUTION

Status: Acknowledged.

H-01 Withdrawal Race Condition Causes Balance DesyncCONTINUED

<https://github.com/sapiencexyz/sapience/pull/672>

HIGH **H-02**

ACKNOWLEDGED

removeWithdrawalIntent() can be abused

CATEGORY	Gaming
LOCATION	BondManagement.sol, MarketLayerZeroBridge.sol
REVIEW	Remediation Review

DESCRIPTION

The recently added `removeWithdrawalIntent()` function in the `BondManagement` contract allows malicious bond depositors to withdraw their bond without only on the UMA side, i.e without reflecting it on the Market side.

Example:

- Alice has `1000` bond balance on both sides
- She initiates a `1000` bond withdrawal
- Her intent chains becomes `1000` on both chains
- She waits 1 day
- Now she can atomically execute `removeWithdrawalIntent()` and `intentToWithdrawBond(amount=1000)`, which will queue 2 LZ messages
- The attacker frontruns the executor and executes the `intentToWithdrawBond` transaction first. It will have no effect on the storage because the intent is already `1000` and the code uses `=`, not `+=`
- Next, the intent removal transaction will reduce the intent to 0.
- When 1 days passes and Alice calls `executeWithdrawal()`, she receives her tokens on the UMA side, but the transaction will revert on the Market side because it will try to subtract 1000 from the 0 intent.

In result Alice withdrew her funds, but the system kept her balance recorded on the Market side.

RECOMMENDATION

Consider:

- enforcing a delay between `removeWithdrawalIntent()` and `intentToWithdrawBond()`
- using `+=` instead of `=` in `MarketLayerZeroBridge._lzReceive()` for the `CMD_FROM_ESCROW_INTENT_TO_WITHDRAW` case.

H-02 `removeWithdrawalIntent()` can be abusedCONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/sapiencexyz/sapience/pull/672>

MEDIUM **M-01**

ACKNOWLEDGED

Dispute Flag Prevents Market Settlement Even After UMA Confirms Truth

CATEGORY Unexpected Behavior
LOCATION UMASettlementModule.sol
REVIEW Remediation Review

DESCRIPTION

In `UMASettlementModule`, once an assertion is disputed the contract sets a sticky flag that blocks automatic settlement even if UMA's final verdict later confirms the assertion as truthful. The critical paths are:

```
// UMASettlementModule.sol
function assertionDisputedCallback(bytes32 assertionId) external nonReentrant {
    ...
    Market.Settlement storage settlement = market.settlement;
    settlement.disputed = true; // <- permanently flipped on first dispute
    emit SettlementDisputed(marketId, block.timestamp);
}

function assertionResolvedCallback(bytes32 assertionId, bool assertedTruthfully) external {
    ...
    // Settles only if the assertion was truthful AND no dispute ever happened
    if (assertedTruthfully && !market.settlement.disputed) {
        market.setSettlementPriceInRange(
            DecimalPrice.sqrtRatioX96ToPrice(settlement.settlementPriceSqrtX96)
        );
        emit MarketSettled(marketId, assertionId, settlement.settlementPriceSqrtX96);
    }
    // Regardless of outcome, the assertion is closed
    market.assertionId = bytes32(0);
}
```

The effect is that any single dispute sets `settlement.disputed = true` and prevents `assertionResolvedCallback` from ever settling the market for that assertion, even when `assertedTruthfully == true` after UMA's DVM resolution. The assertion is then closed (`assertionId` cleared) without settling, leaving the market in the "unsettled" state. Operators must

M-01 Dispute Flag Prevents Market Settlement Even After UMA Confirms Truth **CONTINUED**

re-submit a new assertion and wait through another liveness period to settle. This exposes the system to an application-level griefing vector: a disputer willing to stake UMA's dispute bond can force at least one extra round and extra latency for settlement, even if they ultimately lose on UMA. On bridged deployments, the Market side already deducts the escrowed bond when forwarding the assertion and the UMA side returns bonds directly to the submitter wallet rather than escrow; there is no balance top-up message on "truthful" resolution, so the submitter must re-deposit before the required re-assertion. During this period, positions cannot be finalized because settlement gates `SettlementModule.settlePosition` on `market.settled == true`, so users wait longer than necessary despite UMA having confirmed the truth.

— RECOMMENDATION

If the intention is to trust UMA's final verdict, remove the `!market.settlement.disputed` gate and optionally clear the flag on resolution to reflect state accurately.

If, instead, the business rule is to require human intervention after any dispute, add an explicit post-UMA finalize path that can be called once with UMA's `assertedTruthfully == true`, rather than silently dropping settlement and forcing a full re-assertion cycle.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Innaccurate tradeSlippage Parameter

CATEGORY Unexpected Behavior
LOCATION LiquidityModule.sol
REVIEW Remediation Review

DESCRIPTION

The close paths enforce a “max-loss” check against the collateral baseline rather than bounding the swap cost needed to repay debt. When a position is closed through a `closeLiquidityPosition` call, the quantity that must be controlled is the repayment swap’s execution price, not the final net collateral returned. Because the current guard compares payout to $(1 - \text{tradeSlippage}) \times \text{depositedCollateral}$ and is evaluated after the Uniswap swap, it fails to provide execution-price protection and can allow the swap to clear at a price far worse than the user’s tolerance.

- Swap performed without any slippage:

```
// swap0rQuoteTokensExactOut
ISwapRouter.ExactOutputSingleParams memory swapParams = ISwapRouter.ExactOutputSingleParams({
    tokenIn: tokenIn,
    tokenOut: tokenOut,
    amountOut: amountOut,
    fee: market.marketParams.feeRate,
    recipient: address(this),
    deadline: block.timestamp,
    // Notice, not limiting the trade in any way since we are limiting the collateral required afterwards.
    sqrtPriceLimitX96: 0,
    amountInMaximum: type(uint256).max
});
```

Notice as well the comment “Notice, not limiting the trade in any way since we are limiting the collateral required afterwards”.

- Actual collateral `tradeSlippage` limitation:

```
function _closeTradePosition(Market.Data storage market, Position.Data storage position,
uint256 tradeSlippage) internal
```

M-02 Innacurate tradeSlippage Parameter

CONTINUED

```

{
  int256 deltaCollateralLimit =
    -int256(position.depositedCollateralAmount.mulDecimal(DecimalMath.UNIT - tradeSlippage));
  // Trade & rebalances
  ...
  // Check if the collateral is within the limit
  Trade.checkDeltaCollateralLimit(deltaCollateral, deltaCollateralLimit);
}

```

Proof-of-Concept (PoC): Collateral-anchored "slippage" check lets the same >1% price impact pass or fail depending only on deposited collateral

The test shows that `closeLiquidityPosition` uses `tradeSlippage` as a collateral loss tolerance, not as execution price protection. As a result, the exact same Uniswap swap with ~5.69% slippage (relative to the pre-trade mid) reverts when the LP has "normal" collateral, but succeeds when the LP stuffs more collateral into the position, even though the execution price does not change. This proves the guard is not bounding price impact, it only gates how much of your own collateral the contract is willing to burn.

Test flow at a glance:

1. Setup a market and two LPs: LP1 adds deep, wide liquidity. LP2 adds concentrated liquidity over [-7000, -6800] with 200e18 collateral.
2. Open a 200 base long, nudging the pool from tick -6932 to -6812. At this point the pre-trade mid for the later close is:

```

Current pool price (D18) ..... 506032891067404866
Current pool tick ..... -6812

```

1. Snapshot state, then top up LP2's collateral by 100e18 (to ~300e18) via `increaseLiquidityPosition`, keeping the same borrowed base/quote exposure.
2. Attempt to close with a 1% `tradeSlippage`. It reverts. The revert comes after a Uniswap swap that pushes price to tick -5906.
3. Revert to snapshot, then repeat the same steps but inject much more collateral (the test uses `collateralAmount`: 10000e18 on the increase). Exposure is left the same, only the collateral buffer is larger.
4. Attempt the exact same close with the same 1% `tradeSlippage`. This time it does not revert, even though the identical Uniswap swap executes (same amounts, same final tick), i.e. the same ~5.69% slippage.

M-02 Innacurate tradeSlippage Parameter

CONTINUED

Passing/failing is therefore explained solely by how much collateral was deposited (and the PNL which was constant in both closures), not by the execution price relative to any pre-quote reference.

This is conclusive evidence that the current `tradeSlippage` check is not an execution-price bound and can be bypassed by inflating collateral, exposing users to fills far outside their stated slippage tolerance.

— RECOMMENDATION

Consider documenting this behaviour so traders are aware of this functionality.

`LiquidityCloseParams.amount0Min` and `LiquidityCloseParams.amount1Min` should be enough to mitigate any unwanted slippage.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/6a7a0941cc755a1f4868d6f0fa9f6f62>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

UMALayerZeroBridge Only Supports A Single bondToken

CATEGORY Configuration
LOCATION UMALayerZeroBridge.sol
REVIEW Remediation Review

DESCRIPTION

The `UMALayerZeroBridge` contract is designed to handle bond deposits and assertions for cross-chain UMA settlements but enforces a single enabled bond token via the `_isValidToken` check, which only allows the pre-configured `enabledBondToken`. This creates a logical inconsistency with the broader protocol design, where market groups can be initialized with arbitrary bond currencies (via `MarketGroup.createValid` and `ISapienceStructs.MarketParams.bondCurrency`), the UMA `OptimisticOracleV3` supports multiple currencies (per its `getMinimumBond` for any ERC20), and the `MarketLayerZeroBridge` maintains a flexible mapping (`remoteSubmitterBalances[submitter][bondToken]`) to track balances per token.

As a result, if a market group uses a `bondCurrency != enabledBondToken`, submitters cannot deposit bonds through `UMALayerZeroBridge` (reverts on `depositBond` due to `_isValidToken` failure), preventing assertion submissions and blocking market settlements entirely for those markets. The constructor sets a single `enabledBondToken`, with no mechanism to add more post-deployment.

RECOMMENDATION

Replace the single `enabledBondToken` with a `mapping(address => bool) approvedBondTokens;` add owner-only functions to approve/revoke tokens. Update `_isValidToken` to check the mapping and ensure constructor/market params validate against it.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Incorrect ERC-721 Safe Mint Check Before Minting And With Wrong From Address

CATEGORY Unexpected Behavior
LOCATION LiquidityModule.sol
REVIEW Remediation Review

DESCRIPTION

In the `LiquidityModule`'s `createLiquidityPosition` function, the protocol attempts to ensure safe transfer of the position NFT by calling `_check0nERC721Received` before actually minting the token. This violates the ERC-721 standard, which specifies that for `mint` operations, the `from` address in the `onERC721Received` callback should be `address(0)` and the callback should only be invoked after the `mint` has occurred (i.e., after the token exists and ownership is assigned).

Here, the check is performed prematurely with `from` set to `address(this)` instead of `address(0)`, and the `tokenId` is passed before it's minted. This can lead to inconsistent behavior in recipient contracts that implement `onERC721Received` with logic depending on the `from` parameter or token existence.

RECOMMENDATION

Move the `_check0nERC721Received` call after `_mint` and all the state updates and pass `from` as `address(0)` to align with ERC-721 mint semantics, ensuring the token exists before the callback.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Redundant `intent.executed` Withdrawal Flag

CATEGORY Gas Optimization
LOCATION BondManagement.sol: 143
REVIEW Remediation Review

DESCRIPTION

The `executeWithdrawal` function now sets `intent.amount` to zero when executed, which fully prevents double-execution attempts. This makes the `intent.executed` flag redundant, as it no longer provides any additional protection and only adds unnecessary state writes.

RECOMMENDATION

Remove all instances of `intent.executed` from the codebase

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Dust Liquidity Decrease Can Trigger Revert

CATEGORY Rounding
LOCATION LiquidityModule.sol: 165
REVIEW Remediation Review

DESCRIPTION

When decreasing very small amounts of liquidity (e.g. 100 wei), the operation can revert with an `InsufficientCollateral` error. This occurs due to rounding behavior in Uniswap's `decreaseLiquidity` flow.

In `decreaseLiquidityPosition`, the call to `INonfungiblePositionManager.decreaseLiquidity` expects a non-zero `decreaseAmount0 / decreaseAmount1` which accrue to `stack.tokens0wed`. For dust liquidity amounts, these round down to zero.

Despite receiving no tokens back, the subsequent call to `updateValidLp` reduces the recorded liquidity by the dust amount.

When `requiredCollateralForLiquidity` is called, the remaining liquidity position is valued lower, but without a corresponding increase in `stack.tokens0wed`. This can raise the required collateral above what is already deposited, triggering an `InsufficientCollateral` revert.

This typically occurs after consecutive `decreaseLiquidityPosition` calls, where the first call correctly returns excess collateral, but subsequent dust removals create a mismatch between collateral requirements and accounted tokens.

RECOMMENDATION

Consider introducing a minimum liquidity threshold for `decreaseLiquidityPosition` to prevent operations that would round down to zero. Otherwise, add clear warnings or revert messages so that users are aware of the potential revert condition.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Gas threshold are checked after message is sent

CATEGORY Events
LOCATION UMLayerZeroBridge.sol: 179
REVIEW Remediation Review

— DESCRIPTION

[L-12](https://www.notion.so/guardianaudits/L-12-23e8bda5828c80d4ba4cc45d95acd000?source=copy_link) from the main review points out how `_checkGasThresholds()` is executed before sending the crosschain message in both `MarketLayerZeroBridge` and `UMLayerZeroBridge`. The order of execution was changed in `MarketLayerZeroBridge`, but not in `UMLayerZeroBridge`, which creates a greater inconsistency.

— RECOMMENDATION

Change the order of execution in `UMLayerZeroBridge` as well.

— RESOLUTION

Status: Acknowledged.

<https://github.com/sapiencexyz/sapience/pull/672>

LOW **L-05**

ACKNOWLEDGED

Race Condition May Block The Settlement

CATEGORY	DoS
LOCATION	MarketLayerZeroBridge.sol, UMLayerZeroBridge.sol
REVIEW	Remediation Review

— DESCRIPTION

Even though a remediation link for [M-13](https://www.notion.so/guardianaudits/M-13-23f8bda5828c802eb3c8faa23fe75e78?source=copy_link) was provided, the issue was never fixed and the described scenario can still happen. In fact, it's impact is even bigger now because the function for manual market settlement doesn't exist anymore.

— RECOMMENDATION

You can allow the owner of the `BondManagement` to deposit on behalf of other users. This would require them to pay bond tokens in the described scenario in the issue, but it will save the market from halting.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Invalid Comment About depositCollateral

CATEGORY Documentation
LOCATION Position.sol: 128
REVIEW Remediation Review

— DESCRIPTION

In the function `validateLp`, the comment that states:

```
// called from depositCollateral, that way the check for self.kind is not needed as  
// both trader and lps can deposit collateral as long as they are the owner of fee collector  
NFT
```

is invalid as the `depositCollateral` function has been removed.

— RECOMMENDATION

Update the comment to reflect how it is only called by `preValidateLp`.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-02**

ACKNOWLEDGED

Outdated Comment On Fee Collector

CATEGORY Informational
LOCATION LiquidityModule.sol: 197
REVIEW Remediation Review

DESCRIPTION

In `decreaseLiquidityPosition`, a comment mentions `feeCollector` which has been removed.

RECOMMENDATION

Remove all references to the outdated `feeCollector`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Note about assertedTruthfully

CATEGORY Informational
LOCATION UMASettlementModule.sol
REVIEW Remediation Review

DESCRIPTION

`UMASettlementModule.assertionResolvedCallback()` was changed to respect the `assertedTruthfully` flag which handles the crosschain race condition where an assertion has been disputed but the call to `assertionDisputedCallback()` hasn't been received yet.

Keep in mind that this does not solve the issue with anyone being able to delay the dispute, because `assertionResolvedCallback()` will still skip setting the price if the assertion was disputed (no matter if it was a successful dispute or not)

RECOMMENDATION

Acknowledge it

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.